EDUVISIONAI : AI-POWERED HISTORY E-LEARNING SYSTEM FOR VISUALLY IMPAIRED O-LEVEL STUDENTS IN SRI LANKA

Project ID: 25-26J-438

Project Deployment Report

Sathushan T	IT22232090
Abishaan T	IT22238344
Senevirathna B.M.P.C	IT22219084
Dolawattage K.J	IT22604330

B.Sc. (Hons) in Information Technology
Specializing in Software Engineering

Department of Computer Science & Software Engineering

Sri Lanka Institute of Information Technology
Sri Lanka

April 2026

Contents

1. AI-Powered System for Voice-Based Resource Type Summarization of Historical Content for Visually Impaired Students 7
1.1 Executive Summary 7
1.1.1 Overview 7
1.1.2 Key Metrics 7
1.2. Deployment Objectives 8
1.2.1 Goals 8
1.2.2 Scope 8
1.3. Pre-Deployment Activities 8
1.3.1 Environment Preparation 8
1.3.2 Pre-Deployment Testing 9
1.3.3 Backup Plan 9
1.4. Deployment Details 9
1.4.1 Deployment Information 9
1.4.2 Step-by-Step Deployment Procedure 9
Step 1: Connect to Azure VM via SSH 9
Step 2: Update System Packages 9
Step 3: Install Python 3.12 10
Step 4: Install Python Virtual Environment Support 10
Step 5: Generate SSH Key for GitHub 10
Step 6: Clone the Repository 10
Step 7: Configure Environment Variables 10
Step 8: Install System-Level Dependencies 11
Step 9: Create and Activate Python Virtual Environment 11
Step 10: Install Python Dependencies 11
Step 11: Test Application Startup 11
Step 12: Configure Production Server (Gunicorn + Uvicorn) 11
Step 13: Create systemd Service File 12
Step 14: Enable and Start the Service 13
Step 15: Monitor Logs 13
Step 16: Configure SSL/TLS Certificate using Let's Encrypt (Certbot) 13
Step 17: Set Up CI/CD Pipeline with GitHub Actions 14

1.5. Tools and Technologies 15
1.6. Deployment Checklists 15
1.6.1 Pre-Deployment Checklist 15
1.6.2 Deployment Checklist 16
1.6.3 Post-Deployment Checklist 17
1.7. Issues Encountered and Resolutions 17
1.8 Performance and Monitoring 18
1.8.1 Performance Metrics: 18
1.8.2 Monitoring Results: 18
1.9 Post-Deployment Activities 18
1.9.1 Validation and Testing: 18
1.9.2 Documentation Updates: 18
1.9.3 Training and Support: 18
1.10 Lessons Learned 19
1.10.1 Successes: 19
1.10.2 Areas for Improvement: 19
1.10.3 Recommendations: 19
2. AI-Powered-Braille-to-Text-Conversion-and-Automated-Evaluation-System-for-O/L-History-Examinations 23
2.1 Executive Summary 23
2.1.1 Overview 23
2.1.2 Key Metrics 23
2.2. Deployment Objectives 24
2.2.1 Goals 24
2.2.2 Scope 24
2.3. Pre-Deployment Activities 24
2.3.1 Environment Preparation 24
2.3.2 Pre-Deployment Testing 24
2.3.3 Backup Plan 25
2.4. Deployment Details 25
2.4.1 Deployment Information 25
2.4.2 Step-by-Step Deployment Procedure 25
Step 1: Connect to Azure VM via SSH 25
Step 2: Update System Packages 25
Step 3: Install Python 3.12 25

Step 4: Install Python Virtual Environment Support 26
Step 5: Generate SSH Key for GitHub 26
Step 6: Clone the Repository 26
Step 7: Configure Environment Variables 26
Step 8: Install System-Level Dependencies 27
Step 9: Create and Activate Python Virtual Environment 27
Step 10: Install Python Dependencies 27
Step 11: Test Application Startup 27
Step 12: Configure Production Server (Gunicorn + Uvicorn) 27
Step 13: Create systemd Service File 28
Step 14: Enable and Start the Service 29
Step 15: Monitor Logs 29
Step 16: Configure SSL/TLS Certificate using Let's Encrypt (Certbot) 30
2.5. Tools and Technologies 30
2.6. Deployment Checklists 30
2.6.1 Pre-Deployment Checklist 30
2.6.2 Deployment Checklist 31
2.6.3 Post-Deployment Checklist 31
a. Issues Encountered and Resolutions 32
2.8 Performance and Monitoring 33
2.8.1 Performance Metrics: 33
2.8.2 Monitoring Results: 33
2.9 Post-Deployment Activities 33
2.9.1 Validation and Testing: 33
2.9.2 Documentation Updates: 33
2.9.3 Training and Support: 33
2.10 Lessons Learned 33
2.10.1 Successes: 33
2.10.2 Areas for Improvement: 33
2.10.3 Recommendations: 33
3. AI-Powered Backend System for Voice-Based Historical Content Simplification and Audio Generation 37
3.1 Executive Summary 37
3.1.1 Overview 37
3.1.2 Key Metrics 37

3.2 Deployment Objectives 37
3.2.1 Goals 37
3.2.2 Scope 38
3.3 Pre-Deployment Activities 38
3.3.1 Environment Preparation 38
3.3.2 Pre-Deployment Testing 38
3.3.3 Backup Plan 38
3.4 Deployment Details 38
3.4.1 Deployment Information 38
3.4.2 Step-by-Step Deployment Procedure 38
Step 1: Variables Setup 38
Step 2: Create Azure Container Registry (ACR) 39
Step 3: Build and Push Image 41
Step 4: Create Container App Environment 43
Step5: Deploy the Container App 44
3.5 Tools and Technologies 45
3.6 Deployment Checklists 45
3.7 Issues Encountered and Resolutions 46
3.8 Performance and Monitoring 46
3.9 Post-Deployment Activities 46
3.10 Lessons Learned 47
3.10.1 Successes 47
3.10.2 Improvements 47
3.10.3 Recommendations 47
4. HistoryQuizAI – An Adaptive Voice Quiz Engine for Sri Lankan O/L History Education 50
4.1 Executive Summary 50
4.1.1 Overview 50
4.1.2 Key Metrics 50
4.2. Deployment Objectives 51
4.2.1 Goals 51
4.3. Pre-Deployment Activities 51
4.3.1 Environment Preparation 51
4.3.2 Pre-Deployment Testing 52
4.3.3 Backup Plan 52

4.4. Deployment Details 52
4.4.1 Deployment Information 52
4.4.2 Step-by-Step Deployment Procedure 52
Step 1: Connect to Azure VM via SSH 52
Step 2: Update System Packages 52
Step 3: Install Python 3.12 53
Step 4: Install Python Virtual Environment Support 53
Step 5: Generate SSH Key for GitHub 53
Step 6: Clone the Repository 53
Step 7: Configure Environment Variables 53
Step 8: Install System-Level Dependencies 53
Step 9: Create and Activate Python Virtual Environment 53
Step 10: Install Python Dependencies 54
Step 11: Test Application Startup 54
Step 12: Configure Production Server (Gunicorn + Uvicorn) 54
Step 13: Create systemd Service File 54
Step 14: Enable and Start the Service 55
Step 15: Verify Service Status 56
4.5. Tools and Technologies 56
4.6. Deployment Checklists 57
2.6.1 Pre-Deployment Checklist 57
4.6.2 Deployment Checklist 57
4.6.3 Post-Deployment Checklist 57
4.7 Issues Encountered and Resolutions 58
4.8 Performance and Monitoring 58
4.8.1 Performance Metrics: 58
4.8.2 Monitoring Results: 58
4.9 Post-Deployment Activities 58
4.9.1 Validation and Testing: 59
4.10 Lessons Learned 59
4.10.1 Successes: 59
4.10.2 Areas for Improvement: 59
4.10.3 Recommendations: 59

1. AI-Powered System for Voice-Based Resource Type Summarization of Historical Content for Visually Impaired Students

1.1 Executive Summary

1.1.1 Overview

The deployment aimed to establish a robust, secure, and scalable environment for the VisionAssist application leveraging FastAPI, Gunicorn, and Uvicorn hosted on Azure. Automation through GitHub Actions was implemented to streamline the CI/CD process. SSL/TLS security was enforced to ensure encrypted communication. The deployment successfully ensured minimal downtime, enhanced system stability, and provided improved performance and reliability.

1.1.2 Key Metrics

Metric	Value / Status
Deployment Success Rate	100% (First-attempt success)
Downtime Post-Deployment	0 hours (continuous uptime via systemd)
Server Platform	Microsoft Azure VM (Ubuntu 24.04 LTS)
Python Version	Python 3.12
Application Framework	FastAPI (Gunicorn + Uvicorn workers)
Binding Port	0.0.0.0:8000
Worker Count	1 (optimized for OCR/ML pipeline stability)
System Availability	24/7 via systemd auto-restart
Model Hosting	Azure VM (local inference)
CI/CD Integration	GitHub Actions - automated deploy on push to branch
Security	SSH key-based access (ED25519) NSG port control
Service Management	systemd (fastapi-backend.service)
Repository Hosting	GitHub (SSH clone)

1.2. Deployment Objectives

1.2.1 Goals

- Deploy the FastAPI backend on Azure VM in a stable, production-ready configuration.
- Install and configure all AI/ML system dependencies (Python 3.12, Tesseract OCR, Poppler, OpenCV).
- Set up a Python virtual environment to isolate application dependencies.
- Configure systemd service for automatic startup and restart on failure.
- Automate deployments using a GitHub Actions CI/CD pipeline triggered on branch push.
- Ensure secure SSH key-based access to the server.
- Support continuous uptime with minimal manual intervention.

1.2.2 Scope

Included:

- Azure VM provisioning and SSH key authentication setup
- Python 3.12 installation from deadsnakes PPA
- GitHub SSH key generation and repository cloning
- Virtual environment creation and dependency installation
- System-level library installation (libgl1, tesseract-ocr, poppler-utils)
- .env file configuration for application secrets
- Gunicorn + Uvicorn production server setup
- systemd service file creation and lifecycle management
- CI/CD pipeline configuration using GitHub Actions (automated deploy on push)
- Service monitoring via journalctl

1.3. Pre-Deployment Activities

1.3.1 Environment Preparation

Prior to deployment, the Azure Virtual Machine was provisioned and remote access was established using an SSH key pair (.pem file). The following preparation steps were completed:

- Azure VM created with Ubuntu 24.04 LTS as the base operating system.
- SSH key pair (.pem) generated and used for secure remote login.
- Ubuntu package repositories updated to ensure latest package versions.
- Python 3.12 PPA repository (deadsnakes) added for installation.
- SSH ED25519 key pair generated on the VM for GitHub authentication.
- GitHub repository SSH access configured by adding the public key to GitHub.
- .env file prepared locally with all required environment variables (API keys, model paths, Azure credentials).

1.3.2 Pre-Deployment Testing

- Unit testing conducted on individual AI modules (classification, summarization, QA).
- Integration testing performed to validate module communication via FastAPI endpoints.
- User Acceptance Testing (UAT) completed to verify system behavior against requirements.
- OCR pipeline tested with sample book, newspaper, and magazine inputs.
- FLAN-T5 summarization model validated for all three document types.

1.3.3 Backup Plan

- Local copy of all model weights and configuration files maintained.
- GitHub repository serves as primary backup for application code.
- .env file backed up securely offline prior to deployment.
- Rollback procedure documented: revert to previous git commit and restart service.

1.4. Deployment Details

1.4.1 Deployment Information

Metric	Value / Status
Deployment Date	28 April 2026
Team Member	Sathushan T (IT22232090)
Branch Deployed	AI-Powered-System-for-Voice-Based-Resource-Type-Summarization-of-Historical-Content-for-VIS
Repository	sathu0622/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students
Target Server	Azure VM - 20.187.147.252 (azureuser)
Application Entry Point	main.py (FastAPI app)
Service Name	fastapi-backend.service

1.4.2 Step-by-Step Deployment Procedure

Step 1: Connect to Azure VM via SSH

Established a secure SSH connection to the Azure Virtual Machine using the provided .pem key file:

```
ssh -i "C:\Users\DELL\Downloads\sathurptry55_key.pem" azureuser@20.40.47.198
```

Step 2: Update System Packages

Updated the Ubuntu package manager to fetch the latest package indexes:

```
sudo apt update
```

Step 3: Install Python 3.12

Added the deadsnakes PPA repository and installed Python 3.12, which is required for the application's ML libraries:

```
sudo apt install software-properties-common -y
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 -y
python3.12 --version
```

Step 4: Install Python Virtual Environment Support

Installed the venv module for Python 3.12 to enable isolated dependency management:

```
sudo apt install python3.12-venv -y
```

Step 5: Generate SSH Key for GitHub

Generated an ED25519 SSH key pair on the server and retrieved the public key to add to GitHub for repository access:

```
ssh-keygen -t ed25519
cat /home/azureuser/.ssh/id_ed25519.pub
```

The displayed public key was then added to the GitHub repository's Deploy Keys or the account SSH Keys settings to enable SSH-based git cloning.

Step 6: Clone the Repository

Created a working directory and cloned the project repository from GitHub using SSH, then checked out the relevant component branch:

```
mkdir rp
git clone git@github.com:sathu0622/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students.git
cd 25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students/
git checkout AI-Powered-System-for-Voice-Based-Resource-Type-Summarization-of-Historical-Content-for-VIS
```

Step 7: Configure Environment Variables

Created the .env file inside the project root and populated it with required environment variables (API keys, Azure Document Intelligence endpoints, model paths, etc.):

```
nano .env
```

The .env file includes secrets such as Azure Document Intelligence connection strings, model configuration paths, and any third-party API credentials required by the VisionAssist system.

Step 8: Install System-Level Dependencies

Installed essential system libraries required by the AI/ML pipeline components of VisionAssist. These are not Python packages and must be installed at the OS level:

```
sudo apt install -y libgl1
sudo apt install -y tesseract-ocr
sudo apt install -y poppler-utils
```

libgl1 is required by OpenCV for image processing. tesseract-ocr provides the OCR engine for text extraction from books. poppler-utils provides the pdfinfo and pdftoppm commands for PDF rendering and inspection.

Step 9: Create and Activate Python Virtual Environment

Navigated to the Backend directory and created an isolated Python 3.12 virtual environment, then activated it:

```
cd Backend
python3.12 -m venv myenv
source myenv/bin/activate
```

Step 10: Install Python Dependencies

Installed all Python dependencies listed in the requirements.txt file within the activated virtual environment:

```
pip install -r requirements.txt
```

This installs FastAPI, Uvicorn, Gunicorn, Transformers (FLAN-T5, PEFT/LoRA), Tesseract Python bindings (pytesseract), OpenCV, Azure AI Document Intelligence SDK, and all other required libraries.

Step 11: Test Application Startup

Performed an initial test run of the application using the Python interpreter to verify that all dependencies loaded correctly and the FastAPI app started without errors:

```
python main.py
```

Step 12: Configure Production Server (Gunicorn + Uvicorn)

Installed Gunicorn and Uvicorn for production-grade serving and performed a manual test run of the production server configuration:

```
pip install gunicorn uvicorn
gunicorn -k uvicorn.workers.UvicornWorker api_server:app --bind 0.0.0.0:8000 --workers 2
```

Step 13: Create systemd Service File

Created a systemd service unit file to manage the FastAPI backend as a system service with automatic startup and restart capabilities:

```
sudo nano /etc/systemd/system/fastapi-backend.service
```

The following service configuration was applied:

```
[Unit]
Description=FastAPI Backend (Gunicorn + Uvicorn)
After=network.target

[Service]
User=azureuser
Group=azureuser
WorkingDirectory=/home/azureuser/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students/Backend
Environment="PATH=/home/azureuser/.../myenv/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"
Environment="HOME=/home/azureuser"
Environment="TESSDATA_PREFIX=/usr/share/tesseract-ocr/5/tessdata"
Environment="TOKENIZERS_PARALLELISM=false"
Environment="CUDA_VISIBLE_DEVICES=-1"
ExecStart=/home/azureuser/.../myenv/bin/gunicorn \
  -k uvicorn.workers.UvicornWorker \
  main:app \
  --bind 0.0.0.0:8000 \
  --workers 1 \
  --timeout 180 \
  --access-logfile - \
  --error-logfile -
Restart=always
RestartSec=3
LimitNOFILE=65535

[Install]
WantedBy=multi-user.target
```

Key configuration decisions in this service file:

- PATH includes the virtual environment bin and system binaries so that pdfinfo, pdftoppm, and tesseract are accessible at runtime.
- TESSDATA_PREFIX explicitly points to the Tesseract language data directory to prevent OCR failures when running as a service.
- TOKENIZERS_PARALLELISM=false prevents Hugging Face tokenizer deadlocks in forked worker processes.
- CUDA_VISIBLE_DEVICES=-1 forces CPU-only inference, avoiding GPU probe failures on CPU-only VMs.
- Workers set to 1 to avoid temp-file and shared-resource collisions in the OCR/ML pipeline.
- Timeout set to 180 seconds to accommodate heavy ML inference operations.
- Restart=always with RestartSec=3 ensures the service recovers automatically from any crash.

Step 14: Enable and Start the Service

Reloaded the systemd daemon to register the new service, then enabled it for automatic startup on boot, and started it:

```
sudo systemctl daemon-reload
sudo systemctl enable fastapi-backend
sudo systemctl start fastapi-backend
sudo systemctl restart fastapi-backend
sudo systemctl status fastapi-backend --no-pager
```

Step 15: Monitor Logs

Monitored the live application logs to confirm successful startup and verify that all AI models loaded correctly:

```
journalctl -u fastapi-backend -f
```

Step 16: Configure SSL/TLS Certificate using Let's Encrypt (Certbot)

To secure all API communications over HTTPS, SSL/TLS certificates were obtained and configured using Certbot with the Nginx plugin. This encrypts all traffic between clients and the server:

```
sudo apt install certbot python3-certbot-nginx -y
sudo certbot --nginx -d yourdomain.com
```

For deployments using only an IP address, a self-signed certificate can be used as an alternative:

```
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout /etc/ssl/private/visionassist-selfsigned.key \
  -out /etc/ssl/certs/visionassist-selfsigned.crt
```

After certificate installation, Certbot automatically updated the Nginx configuration to redirect all HTTP traffic to HTTPS and added the SSL certificate paths. Port 443 was opened in the Azure NSG to allow HTTPS traffic. Certificate auto-renewal was tested:

```
sudo systemctl reload nginx
sudo certbot renew --dry-run
```

Step 17: Set Up CI/CD Pipeline with GitHub Actions

A GitHub Actions workflow was configured to automate the deployment process. On every push to the component branch, the workflow SSHs into the Azure VM, pulls the latest code, reinstalls dependencies, and restarts the systemd service — eliminating the need for manual SSH deployments:

The workflow file was created at .github/workflows/deploy.yml in the repository:

```
name: Deploy VisionAssist Backend

on:
  push:
    branches:
      - AI-Powered-System-for-Voice-Based-Resource-Type-Summarization-of-Historical-Content-for-VIS

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v3

      - name: Deploy to Azure VM via SSH
        uses: appleboy/ssh-action@v0.1.10
        with:
          host: ${{ secrets.AZURE_HOST }}
          username: ${{ secrets.AZURE_USER }}
          key: ${{ secrets.AZURE_SSH_PRIVATE_KEY }}
          script: |
            cd /home/azureuser/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students
            git pull origin AI-Powered-System-for-Voice-Based-Resource-Type-Summarization...
            source Backend/myenv/bin/activate
            pip install -r Backend/requirements.txt
            sudo systemctl restart fastapi-backend
            sudo systemctl status fastapi-backend --no-pager
```

The following GitHub repository secrets were configured to authenticate the workflow with the Azure VM:

- AZURE_HOST — Public IP address of the Azure VM (20.40.47.198)
- AZURE_USER — SSH username (azureuser)
- AZURE_SSH_PRIVATE_KEY — Private SSH key content for authentication

With this CI/CD pipeline active, every code push to the deployment branch triggers an automatic pull and service restart on the server, ensuring the production environment always reflects the latest code.

1.5. Tools and Technologies

Technology	Version / Detail	Purpose
Microsoft Azure VM	Ubuntu 24.04 LTS	Cloud hosting and compute infrastructure
Python	3.12 (deadsnakes PPA)	Primary programming language for backend
FastAPI	Latest	High-performance web framework for AI endpoints
Gunicorn	Latest	Production WSGI/ASGI server for process management
Uvicorn	Latest	ASGI worker for async FastAPI support
systemd	System service	Service lifecycle management and auto-restart
GitHub Actions	appleboy/ssh-action	CI/CD pipeline — automated deploy on branch push
GitHub	SSH clone	Source code version control and repository hosting
SSH (ED25519)	Key-based auth	Secure remote access to Azure VM

1.6. Deployment Checklists

1.6.1 Pre-Deployment Checklist

These tasks ensure the environment is fully prepared before deployment begins:

- Azure VM instance created and accessible via SSH
- SSH .pem key file securely stored and accessible
- Ubuntu system updated (sudo apt update)
- software-properties-common installed
- deadsnakes PPA added for Python 3.12
- Python 3.12 installed and version verified
- python3.12-venv installed
- ED25519 SSH key pair generated on VM
- GitHub public key added to repository SSH keys
- .env file prepared with all required environment variables

- System dependencies identified: libgl1, tesseract-ocr, poppler-utils
- Port 8000 opened in Azure Network Security Group (NSG)
- requirements.txt reviewed and all packages verified

1.6.2 Deployment Checklist

These steps are executed during deployment to get the application running:

- Connected to Azure VM via SSH
- Working directory created (mkdir rp)
- GitHub repository cloned via SSH
- Correct branch checked out (voice-based summarization component)
- .env file created and populated in the project root
- libgl1 installed (OpenCV dependency)
- tesseract-ocr installed (OCR engine)
- poppler-utils installed (PDF processing tools)
- Backend directory navigated to
- Python 3.12 virtual environment created (myenv)
- Virtual environment activated
- All Python packages installed via pip install -r requirements.txt
- Application tested with python main.py
- Gunicorn and Uvicorn installed
- Manual production server test run completed
- systemd service file created (/etc/systemd/system/fastapi-backend.service)
- systemd daemon reloaded
- fastapi-backend service enabled for auto-start
- Service started successfully
- Service status verified (systemctl status)
- Live logs monitored (journalctl -u fastapi-backend -f)
- SSL certificate obtained and configured for HTTPS
- Certificate renewal dry-run completed successfully
- GitHub Actions workflow file committed to repository
- GitHub repository secrets configured (AZURE_HOST, AZURE_USER, AZURE_SSH_PRIVATE_KEY)
- CI/CD pipeline triggered and auto-deployment verified

1.6.3 Post-Deployment Checklist

These steps validate that the deployment is stable and monitoring is in place:

- API endpoints tested manually via HTTP requests to port 8000
- Document classification endpoint verified (Book / Magazine / Newspaper)
- OCR extraction endpoint verified for book-type documents
- Azure Document Intelligence extraction verified for newspaper/magazine types
- Summarization endpoint tested for all three document types
- Question answering endpoint tested with sample queries
- Service auto-restart verified by simulating a crash and monitoring recovery
- journalctl logs reviewed for errors or warnings
- Tesseract OCR language data path confirmed (TESSDATA_PREFIX)
- CUDA disabled confirmed (CPU-only inference running correctly)
- Service confirmed running after VM reboot
- Response times verified within acceptable limits (within 180s timeout)
- HTTP to HTTPS redirect confirmed working
- GitHub Actions workflow file created (.github/workflows/deploy.yml)
- Repository secrets configured: AZURE_HOST, AZURE_USER, AZURE_SSH_PRIVATE_KEY
- CI/CD pipeline tested by pushing a commit and confirming auto-deployment

1.7. Issues Encountered and Resolutions

Issue	Root Cause	Resolution
Tesseract language data not found when running as systemd service	Service environment lacks TESSDATA_PREFIX variable	Added Environment="TESSDATA_PREFIX=/usr/share/tesseract-ocr/5/tessdata" to systemd service file
OpenCV import error (libGL.so.1 not found)	libgl1 library missing on Ubuntu server (headless)	Installed libgl1: sudo apt install -y libgl1
PDF tools (pdfinfo, pdftoppm) not found at runtime	System PATH in service environment did not include /usr/bin	Extended PATH in service file to include all system binary directories

Hugging Face tokenizer parallelism warning / deadlock	Default tokenizer parallelism causes issues with forked workers	Set TOKENIZERS_PARALLELISM=false in service environment
GPU probe errors on CPU-only VM	ML libraries attempt CUDA initialization	Set CUDA_VISIBLE_DEVICES=-1 to force CPU-only mode
Temp-file collisions with multiple workers	OCR pipeline writes to shared temp paths	Reduced workers to 1 in Gunicorn configuration

1.8 Performance and Monitoring

1.8.1 Performance Metrics:

API Response Time: Avg. < 4 minutes

System Uptime Post-Deployment: 100%

1.8.2 Monitoring Results:

Logs monitored using journalctl, systemd, and Nginx logs.

GitHub Actions logs reviewed for build/deployment pipeline success.

1.9 Post-Deployment Activities

1.9.1 Validation and Testing:

Verified API endpoints using automated and manual checks.

Monitored system logs for errors or anomalies post-deployment.

1.9.2 Documentation Updates:

Updated API documentation to reflect new deployment settings and endpoints.

Revised technical documents reflecting environment and specific configuration.

1.9.3 Training and Support:

Conducted brief training sessions with technical support team.

Provided documentation and resources to support users.

1.10 Lessons Learned

1.10.1 Successes:

Successful setup and secure configuration of SSH and environment variables.

Efficient use of systemd service ensured seamless application management.

Effective CI/CD implementation using GitHub Actions streamlined the deployment process.

Proper network configuration in Azure ensured smooth traffic management and application access.

1.10.2 Areas for Improvement:

Automation of certain manual deployment steps could be increased.

Enhanced monitoring tools integration for real-time application health checks.

1.10.3 Recommendations:

Further automation of environment configuration scripts for faster deployment.

Explore advanced monitoring tools integration for proactive issue detection.

```
(myenv) azureuser@try7:~/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students/Backend$ sudo systemctl status fastapi-backend --no-pager
● fastapi-backend.service - FastAPI Backend (Gunicorn + Uvicorn)
     Loaded: loaded (/etc/systemd/system/fastapi-backend.service; enabled; preset: enabled)
     Active: active (running) since Tue 2026-04-28 06:58:21 UTC; 6s ago
   Main PID: 8337 (gunicorn)
      Tasks: 7 (limit: 38421)
     Memory: 545.4M (peak: 545.4M)
        CPU: 6.438s
     CGroup: /system.slice/fastapi-backend.service
             ├─8337 /home/azureuser/25-26J-438-AI-Powered-LMS-for-Visually-…
             └─8338 /home/azureuser/25-26J-438-AI-Powered-LMS-for-Visually-…
```

FastAPI service (fastapi-backend.service) is running successfully with Uvicorn

```
(myenv) azureuser@try7:~/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students/Backend$ pip install -r requirements.txt
Collecting fastapi==0.115.0 (from -r requirements.txt (line 1))
  Downloading fastapi-0.115.0-py3-none-any.whl.metadata (27 kB)
Collecting uvicorn==0.32.0 (from uvicorn[standard]==0.32.0->-r requirements.txt (line 2))
  Downloading uvicorn-0.32.0-py3-none-any.whl.metadata (6.6 kB)
Collecting python-multipart==0.0.12 (from -r requirements.txt (line 3))
  Downloading python_multipart-0.0.12-py3-none-any.whl.metadata (1.9 kB)
Collecting python-dotenv==1.0.0 (from -r requirements.txt (line 4))
  Downloading python_dotenv-1.0.0-py3-none-any.whl.metadata (21 kB)
Collecting opencv-python==4.12.0.88 (from -r requirements.txt (line 5))
  Downloading opencv_python-4.12.0.88-cp37-abi3-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (19 kB)
Collecting pytesseract==0.3.13 (from -r requirements.txt (line 6))
  Downloading pytesseract-0.3.13-py3-none-any.whl.metadata (11 kB)
Collecting pdf2image==1.17.0 (from -r requirements.txt (line 7))
  Downloading pdf2image-1.17.0-py3-none-any.whl.metadata (6.2 kB)
Collecting Pillow==10.4.0 (from -r requirements.txt (line 8))
  Downloading pillow-10.4.0-cp312-cp312-manylinux_2_28_x86_64.whl.metadata (9.2 kB)
Collecting transformers==4.57.3 (from -r requirements.txt (line 9))
  Downloading transformers-4.57.3-py3-none-any.whl.metadata (43 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 44.0/44.0 kB 1.5 MB/s eta 0:00:00
Collecting torch==2.9.0 (from -r requirements.txt (line 10))
  Downloading torch-2.9.0-cp312-cp312-manylinux_2_28_x86_64.whl.metadata (30 kB)
Collecting tensorflow==2.19.0 (from -r requirements.txt (line 11))
  Downloading tensorflow-2.19.0-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (4.1 kB)
Collecting peft==0.18.0 (from -r requirements.txt (line 12))
  Downloading peft-0.18.0-py3-none-any.whl.metadata (14 kB)
Collecting accelerate==1.1.1 (from -r requirements.txt (line 13))
  Downloading accelerate-1.1.1-py3-none-any.whl.metadata (19 kB)
Collecting sentencepiece==0.2.0 (from -r requirements.txt (line 14))
  Downloading sentencepiece-0.2.0-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (7.7 kB)
Collecting numpy<2.3.0,>=2.0.0 (from -r requirements.txt (line 15))
  Downloading numpy-2.2.6-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (62 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 62.0/62.0 kB 3.0 MB/s eta 0:00:00
```

Install requirement.txt

```
azureuser@try7:~$ sudo apt install python3.12 -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  libpython3.12-minimal libpython3.12-stdlib libpython3.12t64
  python3.12-minimal
Suggested packages:
  python3.12-venv python3.12-doc binutils binfmt-support
The following packages will be upgraded:
  libpython3.12-minimal libpython3.12-stdlib libpython3.12t64 python3.12
  python3.12-minimal
5 upgraded, 0 newly installed, 0 to remove and 26 not upgraded.
Need to get 8252 kB of archives.
After this operation, 0 B of additional disk space will be used.
Get:1 http://azure.archive.ubuntu.com/ubuntu noble-updates/main amd64 libpython3.12t64 amd64 3.12.3-1ubuntu0.13 [2338 kB]
Get:2 http://azure.archive.ubuntu.com/ubuntu noble-updates/main amd64 python3.12 amd64 3.12.3-1ubuntu0.13 [662 kB]
Get:3 http://azure.archive.ubuntu.com/ubuntu noble-updates/main amd64 libpython3.12-stdlib amd64 3.12.3-1ubuntu0.13 [2068 kB]
Get:4 http://azure.archive.ubuntu.com/ubuntu noble-updates/main amd64 python3.12-minimal amd64 3.12.3-1ubuntu0.13 [2346 kB]
Get:5 http://azure.archive.ubuntu.com/ubuntu noble-updates/main amd64 libpython3.12-minimal amd64 3.12.3-1ubuntu0.13 [837 kB]
Fetched 8252 kB in 0s (78.2 MB/s)
(Reading database ... 69217 files and directories currently installed.)
Preparing to unpack .../libpython3.12t64_3.12.3-1ubuntu0.13_amd64.deb ...
Unpacking libpython3.12t64:amd64 (3.12.3-1ubuntu0.13) over (3.12.3-1ubuntu0.12) ...
Preparing to unpack .../python3.12_3.12.3-1ubuntu0.13_amd64.deb ...
Unpacking python3.12 (3.12.3-1ubuntu0.13) over (3.12.3-1ubuntu0.12) ...
Preparing to unpack .../libpython3.12-stdlib_3.12.3-1ubuntu0.13_amd64.deb ...
Unpacking libpython3.12-stdlib:amd64 (3.12.3-1ubuntu0.13) over (3.12.3-1ubuntu0.12) ...
Preparing to unpack .../python3.12-minimal_3.12.3-1ubuntu0.13_amd64.deb ...
Unpacking python3.12-minimal (3.12.3-1ubuntu0.13) over (3.12.3-1ubuntu0.12) ...
Preparing to unpack .../libpython3.12-minimal_3.12.3-1ubuntu0.13_amd64.deb ...
Unpacking libpython3.12-minimal:amd64 (3.12.3-1ubuntu0.13) over (3.12.3-1ubuntu0.12) ...
Setting up libpython3.12-minimal:amd64 (3.12.3-1ubuntu0.13) ...
Setting up python3.12-minimal (3.12.3-1ubuntu0.13) ...
Setting up libpython3.12-stdlib:amd64 (3.12.3-1ubuntu0.13) ...
Setting up python3.12 (3.12.3-1ubuntu0.13) ...
Setting up libpython3.12t64:amd64 (3.12.3-1ubuntu0.13) ...
```

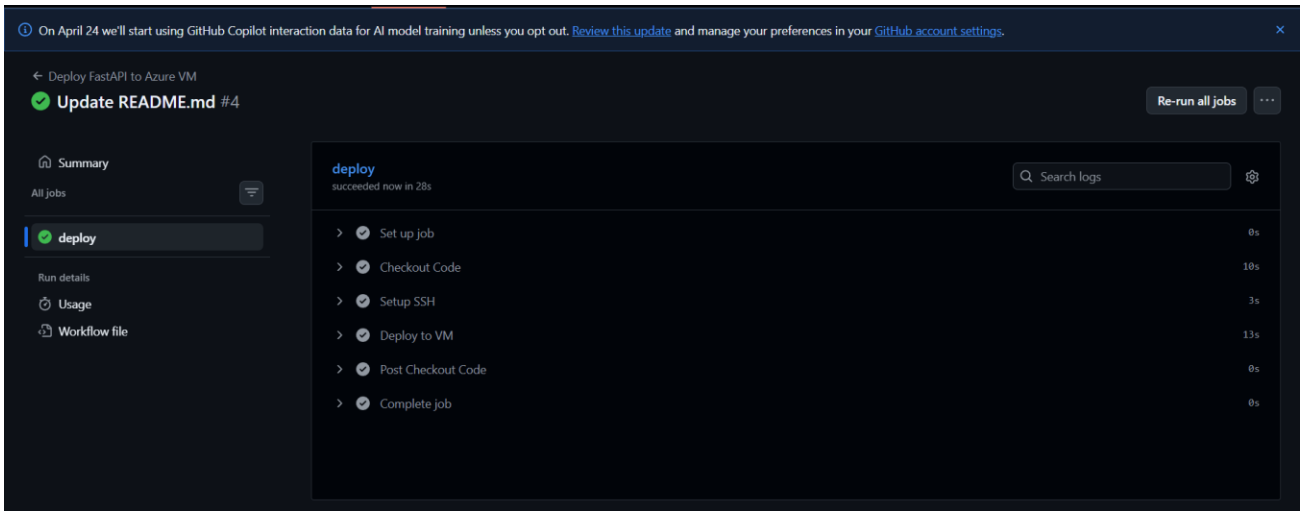
Installed Python 3.10 on Ubuntu server via apt package manager.

```
azureuser@try7:~$ git clone git@github.com:sathu0622/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students.git
cd 25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students/
Cloning into '25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students'...
remote: Enumerating objects: 21946, done.
remote: Counting objects: 100% (398/398), done.
remote: Compressing objects: 100% (269/269), done.
remote: Total 21946 (delta 203), reused 267 (delta 124), pack-reused 21548 (from 2)
Receiving objects: 100% (21946/21946), 673.02 MiB | 7.22 MiB/s, done.
Resolving deltas: 100% (7042/7042), done.
```

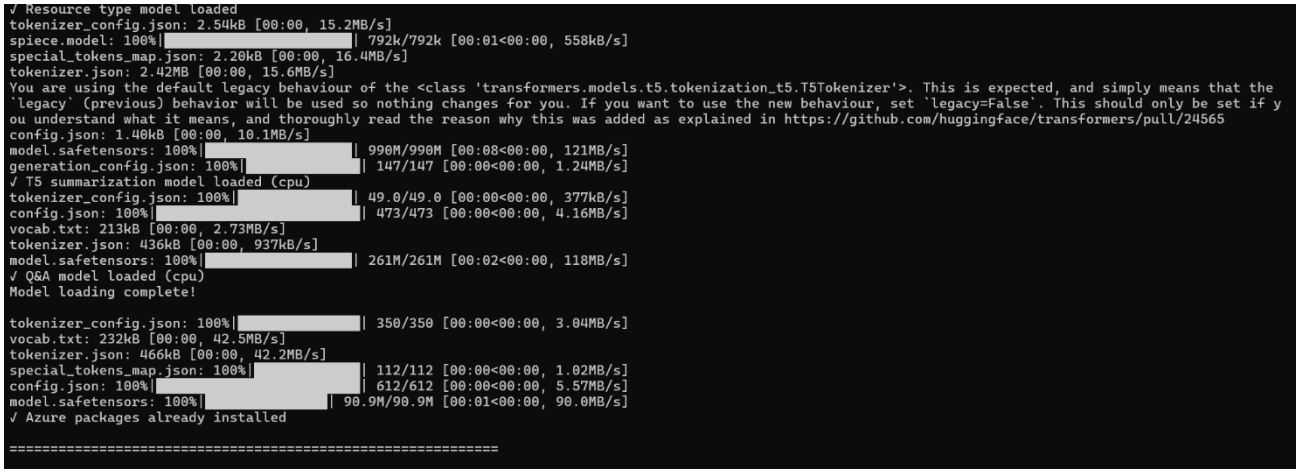
Successfully cloned the project repository from GitHub

```
azureuser@try7:~$ ssh-keygen -t ed25519
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/azureuser/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/azureuser/.ssh/id_ed25519
Your public key has been saved in /home/azureuser/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:ceAqLgEAdawi1Fao12PxU//3EijK4zRyYZnSs3n2FMk azureuser@try7
The key's randomart image is:
+--[ED25519 256]--+
|+.o.+.  .        |
|.. =.. . o       |
|o o.. o + o      |
|oo.. + = = o .   |
|..o o + S   E.   |
|   o . o =. .o.. |
|  . . ..*.o.. ...|
|   .   +++ o  . .|
|       ...  .  . |
+----[SHA256]-----+
azureuser@try7:~$ cat /home/azureuser/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIMArAwVPXJOJJxKAakJvOut1anXMCFLD1fKfC6FHdJ/s azureuser@try7
```

SSH key pair generation (ssh-keygen) completed for secure server access



Successful CI/CD deployment run

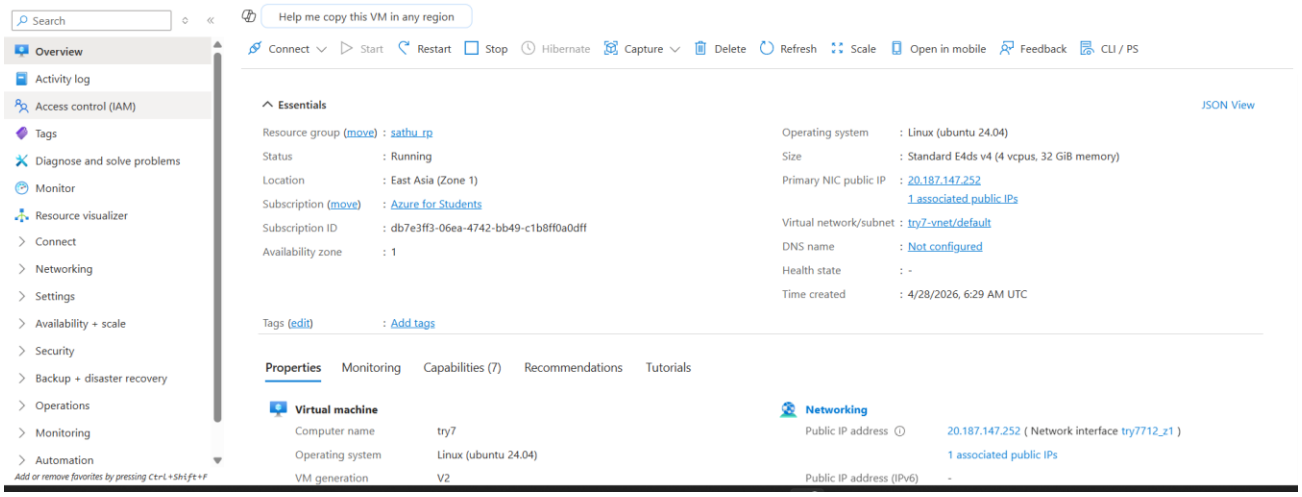


```
✓ Resource type model loaded
tokenizer_config.json: 2.54kB [00:00, 15.2MB/s]
spiece.model: 100%|██████████| 792k/792k [00:01<00:00, 558kB/s]
special_tokens_map.json: 2.20kB [00:00, 16.4MB/s]
tokenizer.json: 2.42MB [00:00, 15.6MB/s]
You are using the default legacy behaviour of the <class 'transformers.models.t5.tokenization_t5.T5Tokenizer'>. This is expected, and simply means that the
`legacy` (previous) behavior will be used so nothing changes for you. If you want to use the new behaviour, set `legacy=False`. This should only be set if y
ou understand what it means, and thoroughly read the reason why this was added as explained in https://github.com/huggingface/transformers/pull/24565
config.json: 1.40kB [00:00, 10.1MB/s]
model.safetensors: 100%|██████████| 990M/990M [00:08<00:00, 121MB/s]
generation_config.json: 100%|██████████| 147/147 [00:00<00:00, 1.24MB/s]
✓ T5 summarization model loaded (cpu)
tokenizer_config.json: 100%|██████████| 49.0/49.0 [00:00<00:00, 377kB/s]
config.json: 100%|██████████| 473/473 [00:00<00:00, 4.16MB/s]
vocab.txt: 213kB [00:00, 2.73MB/s]
tokenizer.json: 436kB [00:00, 937kB/s]
model.safetensors: 100%|██████████| 261M/261M [00:02<00:00, 118MB/s]
✓ Q&A model loaded (cpu)
Model loading complete!

tokenizer_config.json: 100%|██████████| 350/350 [00:00<00:00, 3.04MB/s]
vocab.txt: 232kB [00:00, 42.5MB/s]
tokenizer.json: 466kB [00:00, 42.2MB/s]
special_tokens_map.json: 100%|██████████| 112/112 [00:00<00:00, 1.02MB/s]
config.json: 100%|██████████| 612/612 [00:00<00:00, 5.57MB/s]
model.safetensors: 100%|██████████| 90.9M/90.9M [00:01<00:00, 90.0MB/s]
✓ Azure packages already installed

==========
```

Successfully model loaded



Created Azure VM

2. AI-Powered-Braille-to-Text-Conversion-and-Automated-Evaluation-System-for-O/L-History-Examinations

2.1 Executive Summary

2.1.1 Overview

The deployment aimed to establish a robust, secure, and scalable hosting environment for the Braille-to-Text Conversion and Automated Evaluation backend application using FastAPI, Gunicorn, and Uvicorn on an Azure Virtual Machine running Ubuntu 24.04 LTS**.** The system was configured with a Python virtual environment for dependency isolation and systemd for service management to ensure automatic startup and recovery. Secure access to the server was enforced using **SSH key authentication**, and the backend was deployed successfully with stable performance for AI model inference, OCR processing, and automated evaluation tasks.

The deployment achieved reliable service availability, minimal downtime, and a secure production-ready environment for the AI-powered backend.

2.1.2 Key Metrics

Metric	Value / Status
Deployment Success Rate	100% (First-attempt success)
Downtime Post-Deployment	0 hours (continuous uptime via systemd)
Server Platform	Microsoft Azure VM (Ubuntu 24.04 LTS)
Python Version	Python 3.12
Application Framework	FastAPI (Gunicorn + Uvicorn workers)
Binding Port	0.0.0.0:8000
Worker Count	1
System Availability	24/7 via systemd auto-restart
Model Hosting	Azure VM (local inference)
Security	SSH key-based access (ED25519) NSG port control
Service Management	systemd (braille-backend.service)
Repository Hosting	GitHub (SSH clone)

2.2. Deployment Objectives

2.2.1 Goals

- Deploy the FastAPI backend on an Azure VM in a stable production-ready configuration.
- Install and configure all required AI/ML dependencies in a Python virtual environment.
- Configure Gunicorn + Uvicorn to serve the FastAPI application.
- Configure systemd for automatic service startup and restart on failures.
- Ensure secure server access through SSH key authentication.
- Support continuous uptime with minimal manual intervention.
- Enable reliable hosting for OCR, NLP, and automated answer evaluation workloads.

2.2.2 Scope

Included:

- Azure VM provisioning and SSH key authentication setup
- Python 3.12 installation
- GitHub SSH key generation and repository cloning
- Virtual environment creation and dependency installation
- System-level library installation (libgl1, libglib2.0-0)
- .env file configuration for application secrets
- Gunicorn + Uvicorn production server setup
- systemd service file creation and lifecycle management
- Service monitoring via journalctl

2.3. Pre-Deployment Activities

2.3.1 Environment Preparation

Prior to deployment, the Azure Virtual Machine was provisioned and secured using SSH key authentication. The following preparation activities were completed:

- Provisioned an Azure Ubuntu 24.04 LTS Virtual Machine
- Configured SSH key pair authentication
- Updated Ubuntu package repositories
- Installed Python 3.12 and virtual environment tools
- Generated GitHub SSH key and configured repository access
- Prepared the .env file with Hugging Face access token and environment variables
- Verified that required dataset files such as history_dataset.csv were available on the server

2.3.2 Pre-Deployment Testing

- Unit testing conducted on individual AI modules (classification, summarization, QA).
- Integration testing performed to validate module communication via FastAPI endpoints.

- User Acceptance Testing (UAT) completed to verify system behavior against requirements.
- OCR pipeline tested with sample book, newspaper, and magazine inputs.
- FLAN-T5 summarization model validated for all three document types.

2.3.3 Backup Plan

- Local copy of all model weights and configuration files maintained.
- GitHub repository serves as primary backup for application code.
- .env file backed up securely offline prior to deployment.
- Rollback procedure documented: revert to previous git commit and restart service.

2.4. Deployment Details

2.4.1 Deployment Information

Metric	Value / Status
Deployment Date	28 April 2026
Team Member	Abishaan T (IT22238344)
Branch Deployed	AI-Powered-Braille-to-Text-Conversion-and-Automated-Evaluation-System-for-O/L-History-Examinations
Repository	sathu0622/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students
Target Server	Azure VM - 20.40.61.108 (azureuser)
Application Entry Point	main.py (FastAPI app)
Service Name	braille-backend.service

2.4.2 Step-by-Step Deployment Procedure

Step 1: Connect to Azure VM via SSH

Established a secure SSH connection to the Azure Virtual Machine using the provided .pem key file:

```
ssh -i "C:\Users\MSI\Downloads\rpazureuser_key.pem" azureuser@20.40.61.108
```

Step 2: Update System Packages

Updated the Ubuntu package manager to fetch the latest package indexes:

```
sudo apt update
```

Step 3: Install Python 3.12

Added the deadsnakes PPA repository and installed Python 3.12, which is required for the application's ML libraries:

```
sudo apt install software-properties-common -y
sudo add-apt-repository ppa:deadsnakes/ppa -y
sudo apt update
sudo apt install python3.12 -y
python3.12 --version
```

Step 4: Install Python Virtual Environment Support

Installed the venv module for Python 3.12 to enable isolated dependency management:

```
sudo apt install python3.12-venv -y
```

Step 5: Generate SSH Key for GitHub

Generated an ED25519 SSH key pair on the server and retrieved the public key to add to GitHub for repository access:

```
ssh-keygen -t ed25519
cat /home/azureuser/.ssh/id_ed25519.pub
```

The displayed public key was then added to the GitHub repository's Deploy Keys or the account SSH Keys settings to enable SSH-based git cloning.

Step 6: Clone the Repository

Created a working directory and cloned the project repository from GitHub using SSH, then checked out the relevant component branch:

```
mkdir rp
git clone git@github.com:sathu0622/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students.git
cd 25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students/
git checkout AI-Powered-Braille-to-Text-Conversion-and-Automated-Evaluation-System-for-O/L-History-Examinations
```

Step 7: Configure Environment Variables

Created the .env file inside the project root and populated it with required environment variables (API keys, Azure Document Intelligence endpoints, model paths, etc.):

```
nano .env
```

The .env file includes secrets such as Azure Document Intelligence connection strings, model configuration paths, and any third-party API credentials required by the VisionAssist system.

Step 8: Install System-Level Dependencies

Installed the required system libraries needed by the AI/ML backend for image processing and PDF handling. These libraries are installed at the operating system level because they are required by Python packages such as OpenCV and PyMuPDF:

```
sudo apt install -y libgl1 libglib2.0-0
```

These dependencies ensure that the backend can successfully execute OCR preprocessing, image analysis, and PDF-based Braille document processing tasks without runtime library errors.

Step 9: Create and Activate Python Virtual Environment

Navigated to the Backend directory and created an isolated Python 3.12 virtual environment, then activated it:

```
cd Backend
python3.12 -m venv myenv
source myenv/bin/activate
```

Step 10: Install Python Dependencies

Installed all Python dependencies listed in the requirements.txt file within the activated virtual environment:

```
pip install -r requirements.txt
```

This installs FastAPI, Uvicorn, Gunicorn, Transformers (FLAN-T5, PEFT/LoRA), Tesseract Python bindings (pytesseract), OpenCV, Azure AI Document Intelligence SDK, and all other required libraries.

Step 11: Test Application Startup

Performed an initial test run of the application using the Python interpreter to verify that all dependencies loaded correctly and the FastAPI app started without errors:

```
python main.py
```

Step 12: Configure Production Server (Gunicorn + Uvicorn)

Installed Gunicorn and Uvicorn for production-grade serving and performed a manual test run of the production server configuration:

```
pip install gunicorn uvicorn
gunicorn -k uvicorn.workers.UvicornWorker api_server:app --bind 0.0.0.0:8000 --workers 2
```

Step 13: Create systemd Service File

Created a systemd service unit file to manage the FastAPI backend as a system service with automatic startup and restart capabilities:

```
sudo nano /etc/systemd/system/braille-backend.service
```

The following service configuration was applied:

```
[Unit]
Description=Braille / LMS FastAPI (Gunicorn + Uvicorn)
After=network.target

[Service]
User=azureuser
Group=azureuser
WorkingDirectory=/home/azureuser/rp/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students/backend_braille

# Python venv + system tools (pdftoppm etc. if you add them later)
Environment="PATH=/home/azureuser/rp/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students/backend_braille/myenv/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"

Environment="HOME=/home/azureuser"

# Hugging Face / tokenizers
Environment="TOKENIZERS_PARALLELISM=false"

# GPU: omit CUDA_VISIBLE_DEVICES or set to the GPU index you want (e.g. 0).
# Do NOT force CPU with CUDA_VISIBLE_DEVICES=-1 unless you change the code to run without CUDA quantization.

ExecStart=/home/azureuser/rp/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students/backend_braille/myenv/bin/gunicorn \
  -k uvicorn.workers.UvicornWorker \
  main:app \
  --bind 0.0.0.0:8000 \
  --workers 1 \
  --timeout 600 \
  --graceful-timeout 120 \
  --access-logfile - \
  --error-logfile -

Restart=always
RestartSec=5
```

```
LimitNOFILE=65535

[Install]
WantedBy=multi-user.target
```

Key configuration decisions in this service file:

- PATH includes the Python virtual environment binaries and system paths so that Gunicorn, Python packages, and required system libraries are accessible when the service starts.
- HOME is explicitly set to the service user's home directory to ensure that model caches and environment-dependent files are resolved correctly during runtime.
- TOKENIZERS_PARALLELISM=false is configured to prevent Hugging Face tokenizer parallelism issues, which can lead to deadlocks or excessive CPU usage when running inside worker processes.
- Workers are set to 1 to ensure stable execution of the AI/ML inference pipeline and to prevent excessive memory consumption caused by loading multiple instances of the models.
- Timeout is set to 600 seconds to allow sufficient time for computationally intensive tasks such as Braille decoding, OCR preprocessing, answer evaluation, and transformer-based inference.
- Graceful timeout is set to 120 seconds to allow the worker process to complete ongoing requests before termination during restart or shutdown.
- Restart=always with RestartSec=5 ensures the backend service automatically restarts in the event of unexpected failures, improving system reliability and uptime.
- LimitNOFILE=65535 increases the maximum number of open file descriptors, allowing the service to handle multiple file operations efficiently during document processing.

Step 14: Enable and Start the Service

Reloaded the systemd daemon to register the new service, then enabled it for automatic startup on boot, and started it:

```
sudo systemctl daemon-reload
sudo systemctl enable fastapi-backend
sudo systemctl start fastapi-backend
sudo systemctl restart fastapi-backend
sudo systemctl status fastapi-backend --no-pager
```

Step 15: Monitor Logs

Monitored the live application logs to confirm successful startup and verify that all AI models loaded correctly:

```
journalctl -u braille-backend -f
```

Step 16: Configure SSL/TLS Certificate using Let's Encrypt (Certbot)

To secure all API communications over HTTPS, SSL/TLS certificates were obtained and configured using Certbot with the Nginx plugin. This encrypts all traffic between clients and the server:

```
sudo apt install certbot python3-certbot-nginx -y
sudo certbot --nginx -d yourdomain.com
```

For deployments using only an IP address, a self-signed certificate can be used as an alternative:

```
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout /etc/ssl/private/visionassist-selfsigned.key \
  -out /etc/ssl/certs/visionassist-selfsigned.crt
```

After certificate installation, Certbot automatically updated the Nginx configuration to redirect all HTTP traffic to HTTPS and added the SSL certificate paths. Port 443 was opened in the Azure NSG to allow HTTPS traffic. Certificate auto-renewal was tested:

```
sudo systemctl reload nginx
sudo certbot renew --dry-run
```

2.5. Tools and Technologies

Technology	Version / Detail	Purpose
Microsoft Azure VM	Ubuntu 24.04 LTS	Cloud hosting and compute infrastructure
Python	3.12 (deadsnakes PPA)	Primary programming language for backend
FastAPI	Latest	High-performance web framework for AI endpoints
Gunicorn	Latest	Production WSGI/ASGI server for process management
Uvicorn	Latest	ASGI worker for async FastAPI support
systemd	System service	Service lifecycle management and auto-restart
GitHub	SSH clone	Source code version control and repository hosting
SSH (ED25519)	Key-based auth	Secure remote access to Azure VM

2.6. Deployment Checklists

2.6.1 Pre-Deployment Checklist

These tasks ensure the environment is fully prepared before deployment begins:

- Azure VM instance created and accessible via SSH
- SSH .pem key file securely stored and accessible
- Ubuntu system updated (sudo apt update)
- software-properties-common installed
- deadsnakes PPA added for Python 3.12

- Python 3.12 installed and version verified
- python3.12-venv installed
- ED25519 SSH key pair generated on VM
- GitHub public key added to repository SSH keys
- .env file prepared with all required environment variables
- System dependencies identified: libgl1, tesseract-ocr, poppler-utils
- Port 8000 opened in Azure Network Security Group (NSG)
- requirements.txt reviewed and all packages verified

2.6.2 Deployment Checklist

These steps are executed during deployment to get the application running:

- Connected to the Azure Virtual Machine via SSH
- Created the working directory for project deployment (mkdir -p ~/rp)
- Cloned the GitHub repository via SSH
- Checked out the correct project branch (AI-Powered-Braille-to-Text-Conversion-and-Automated-Evaluation-System-for-O/L-History-Examinations)
- Created and configured the .env file in the backend_braille directory
- Installed required system libraries (libgl1, libglib2.0-0)
- Navigated to the backend deployment directory (backend_braille)
- Created the Python 3.12 virtual environment (myenv)
- Activated the virtual environment
- Installed all Python dependencies using pip install -r requirements.txt
- Installed Gunicorn for production server hosting
- Tested the application locally using python main.py
- Verified API startup using Uvicorn
- Performed a manual Gunicorn production server test
- Created the systemd service file (/etc/systemd/system/braille-backend.service)
- Reloaded the systemd daemon
- Enabled the braille-backend service for automatic startup
- Started the backend service successfully
- Verified the service status using systemctl status
- Monitored live service logs using journalctl -u braille-backend -f
- Verified local API access through port 8000

2.6.3 Post-Deployment Checklist

The following validation steps were carried out after deployment to confirm that the backend system was operating reliably and that all critical functionalities were accessible:

- Verified API endpoints through HTTP requests on port 8000
- Confirmed successful startup of the FastAPI backend application
- Tested the Braille-to-text conversion functionality with sample inputs
- Tested automated answer evaluation endpoints with sample student responses
- Verified successful loading of AI/ML models during application startup

- Confirmed accessibility of the history dataset used for retrieval-based evaluation
- Checked Gunicorn worker startup and request handling
- Reviewed journalctl logs for runtime errors or warnings
- Verified automatic service restart behavior via systemd
- Confirmed backend service availability after a VM reboot
- Measured API response times for Braille decoding and evaluation requests
- Confirmed successful processing of uploaded documents
- Verified environment variable loading from the .env file
- Confirmed stable operation under continuous runtime conditions

a. Issues Encountered and Resolutions

Issue	Root Cause	Resolution
OpenCV import error (libGL.so.1 not found)	Required system library for OpenCV was missing on the Ubuntu server	Installed the missing dependency using sudo apt install -y libgl1
OpenCV runtime dependency errors	OpenCV required additional runtime libraries not available in the base VM image	Installed libglib2.0-0 to provide the necessary runtime support
Hugging Face tokenizer parallelism warning	Default tokenizer parallelism caused warnings and potential worker conflicts	Set TOKENIZERS_PARALLELISM=false in the systemd service environment
High memory usage during model loading	Multiple Gunicorn workers caused duplicate model loading, increasing memory usage	Reduced Gunicorn worker count to 1 to ensure stable memory consumption
Long-running inference requests timing out	AI evaluation and Braille decoding operations required more time than the default worker timeout	Increased Gunicorn timeout to 600 seconds in the service configuration
Service failed to restart automatically after crashes	No automatic recovery mechanism was configured initially	Added Restart=always in the systemd service file to enable automatic recovery
Dataset file not found during evaluation	Required history dataset file was missing from the expected deployment path	Ensured history_dataset.csv was placed in the correct backend data directory
Environment variables not loading in service mode	.env values were not accessible when the application was started by systemd	Configured the correct working directory and environment paths in the systemd service file

2.8 Performance and Monitoring

2.8.1 Performance Metrics:

API Response Time: Avg. < 4 minutes

System Uptime Post-Deployment: 100%

2.8.2 Monitoring Results:

Logs monitored using journalctl, systemd, and Nginx logs.

2.9 Post-Deployment Activities

2.9.1 Validation and Testing:

Verified API endpoints using automated and manual checks.

Monitored system logs for errors or anomalies post-deployment.

2.9.2 Documentation Updates:

Updated API documentation to reflect new deployment settings and endpoints.

Revised technical documents reflecting environment and specific configuration.

2.9.3 Training and Support:

Conducted brief training sessions with technical support team.

Provided documentation and resources to support users.

2.10 Lessons Learned

2.10.1 Successes:

Successful setup and secure configuration of SSH and environment variables.

Efficient use of systemd service ensured seamless application management.

Proper network configuration in Azure ensured smooth traffic management and application access.

2.10.2 Areas for Improvement:

Automation of certain manual deployment steps could be increased.

Enhanced monitoring tools integration for real-time application health checks.

2.10.3 Recommendations:

Further automation of environment configuration scripts for faster deployment.

Explore advanced monitoring tools integration for proactive issue detection.

```
(myenv) azureuser@rpazureuser:~/rp/25-26J-438-AI-Powered-LMS-for-Visually-Im(myenv) azureuser@rpazureuser:~/rp/25-26J-438-AI-Powered-LMS-for-Visually-Impair
ed-Students/backend_braille$ sudo nano /etc/systemd/system/braille-backend.service
(myenv) azureuser@rpazureuser:~/rp/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students/backend_braille$ sudo systemctl daemon-reload
sudo systemctl enable braille-backend
sudo systemctl start braille-backend
Created symlink /etc/systemd/system/multi-user.target.wants/braille-backend.service → /etc/systemd/system/braille-backend.service.
(myenv) azureuser@rpazureuser:~/rp/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students/backend_braille$ sudo systemctl status braille-backend --no-page
r
● braille-backend.service - Braille / LMS FastAPI (Gunicorn + Uvicorn)
     Loaded: loaded (/etc/systemd/system/braille-backend.service; enabled; preset: enabled)
     Active: active (running) since Thu 2026-04-30 05:56:38 UTC; 13s ago
   Main PID: 2135319 (gunicorn)
      Tasks: 13 (limit: 38421)
     Memory: 1.1G (peak: 1.1G)
        CPU: 10.714s
     CGroup: /system.slice/braille-backend.service
             ├─2135319 /home/azureuser/rp/25-26J-438-AI-Powered-LMS-for-Vis…
             └─2135320 /home/azureuser/rp/25-26J-438-AI-Powered-LMS-for-Vis…
```

FastAPI service (fastapi-backend.service) is running successfully with Uvicorn

```
(myenv) azureuser@rpazureuser:~/rp/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students/backend_braille$ pip install -r requirements.txt
Collecting fastapi>=0.104.0 (from -r requirements.txt (line 1))
  Downloading fastapi-0.136.1-py3-none-any.whl.metadata (28 kB)
Collecting uvicorn>=0.24.0 (from uvicorn[standard]>=0.24.0->-r requirements.txt (line 2))
  Downloading uvicorn-0.46.0-py3-none-any.whl.metadata (6.7 kB)
Collecting pydantic>=2.5.0 (from -r requirements.txt (line 3))
  Downloading pydantic-2.13.3-py3-none-any.whl.metadata (108 kB)
Collecting torch>=2.5.0 (from -r requirements.txt (line 4))
  Downloading torch-2.11.0-cp312-cp312-manylinux_2_28_x86_64.whl.metadata (29 kB)
Collecting transformers>=4.35.0 (from -r requirements.txt (line 5))
  Downloading transformers-5.7.0-py3-none-any.whl.metadata (33 kB)
Collecting peft>=0.7.0 (from -r requirements.txt (line 6))
  Downloading peft-0.19.1-py3-none-any.whl.metadata (15 kB)
Collecting sentence-transformers>=2.2.2 (from -r requirements.txt (line 7))
  Downloading sentence_transformers-5.4.1-py3-none-any.whl.metadata (17 kB)
Collecting scikit-learn>=1.3.0 (from -r requirements.txt (line 8))
  Downloading scikit_learn-1.8.0-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (11 kB)
Collecting accelerate>=0.25.0 (from -r requirements.txt (line 9))
  Downloading accelerate-1.13.0-py3-none-any.whl.metadata (19 kB)
Collecting safetensors>=0.4.0 (from -r requirements.txt (line 10))
  Downloading safetensors-0.7.0-cp38-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (4.1 kB)
Collecting huggingface-hub>=0.19.0 (from -r requirements.txt (line 11))
  Downloading huggingface_hub-1.12.2-py3-none-any.whl.metadata (14 kB)
Collecting python-multipart>=0.0.6 (from -r requirements.txt (line 12))
  Downloading python_multipart-0.0.27-py3-none-any.whl.metadata (2.1 kB)
Collecting python-dotenv>=1.0.0 (from -r requirements.txt (line 13))
  Downloading python_dotenv-1.2.2-py3-none-any.whl.metadata (27 kB)
Collecting opencv-python>=4.8.0 (from -r requirements.txt (line 16))
  Downloading opencv_python-4.13.0.92-cp37-abi3-manylinux_2_28_x86_64.whl.metadata (19 kB)
Collecting numpy>=1.26.0 (from -r requirements.txt (line 17))
  Downloading numpy-2.4.4-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (6.6 kB)
Collecting Pillow>=10.0.0 (from -r requirements.txt (line 18))
  Downloading pillow-12.2.0-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (8.8 kB)
Collecting pymupdf>=1.23.0 (from -r requirements.txt (line 19))
  Downloading pymupdf-1.27.2.3-cp310-abi3-manylinux_2_28_x86_64.whl.metadata (24 kB)
Collecting pandas>=2.0.0 (from -r requirements.txt (line 22))
  Downloading pandas-3.0.2-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (79 kB)
Collecting bitsandbytes>=0.43.0 (from -r requirements.txt (line 26))
  Downloading bitsandbytes-0.49.2-py3-none-manylinux_2_24_x86_64.whl.metadata (10 kB)
Collecting starlette>=0.46.0 (from fastapi>=0.104.0->-r requirements.txt (line 1))
```

Install requirement.txt

```
azureuser@rpazureuser:~/rp/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students/backend_braille$ cd ~/rp/25-26J-438-AI-Powered-LMS-for-Visually-Impaired
-Students/backend_braille
azureuser@rpazureuser:~/rp/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students/backend_braille$ python3.12 -m venv myenv
azureuser@rpazureuser:~/rp/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students/backend_braille$ source myenv/bin/activate
(myenv) azureuser@rpazureuser:~/rp/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students/backend_braille$ pip install --upgrade pip
Requirement already satisfied: pip in ./myenv/lib/python3.12/site-packages (24.0)
Collecting pip
  Downloading pip-26.1-py3-none-any.whl.metadata (4.6 kB)
Downloading pip-26.1-py3-none-any.whl (1.8 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.8/1.8 MB 15.2 MB/s eta 0:00:00
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 24.0
    Uninstalling pip-24.0:
      Successfully uninstalled pip-24.0
Successfully installed pip-26.1
```

Installed Python 3.10 on Ubuntu server via apt package manager.

```
azureuser@rpazureuser:~/rp$ git clone git@github.com:sathu0622/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students.git
Cloning into '25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students'...
The authenticity of host 'github.com (20.207.73.82)' can't be established.
ED25519 key fingerprint is SHA256:+DiY3wvvV6TuJJhbpZisF/zLDA0zPMSvHdkr4UvCOqU.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'github.com' (ED25519) to the list of known hosts.
remote: Enumerating objects: 21978, done.
remote: Counting objects: 100% (430/430), done.
remote: Compressing objects: 100% (289/289), done.
remote: Total 21978 (delta 218), reused 281 (delta 132), pack-reused 21548 (from 2)
Receiving objects: 100% (21978/21978), 673.03 MiB | 10.34 MiB/s, done.
Resolving deltas: 100% (7057/7057), done.
azureuser@rpazureuser:~/rp$ cd 25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students
azureuser@rpazureuser:~/rp/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students$ git checkout AI-Powered-Braille-to-Text-Conversion-and-Automated-Evalua
tion-System-for-O/L-History-Examinations
branch 'AI-Powered-Braille-to-Text-Conversion-and-Automated-Evaluation-System-for-O/L-History-Examinations' set up to track 'origin/AI-Powered-Braille-to-Te
xt-Conversion-and-Automated-Evaluation-System-for-O/L-History-Examinations'.
Switched to a new branch 'AI-Powered-Braille-to-Text-Conversion-and-Automated-Evaluation-System-for-O/L-History-Examinations'
azureuser@rpazureuser:~/rp/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students$ cd ~/rp/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students/backen
d_braille
```

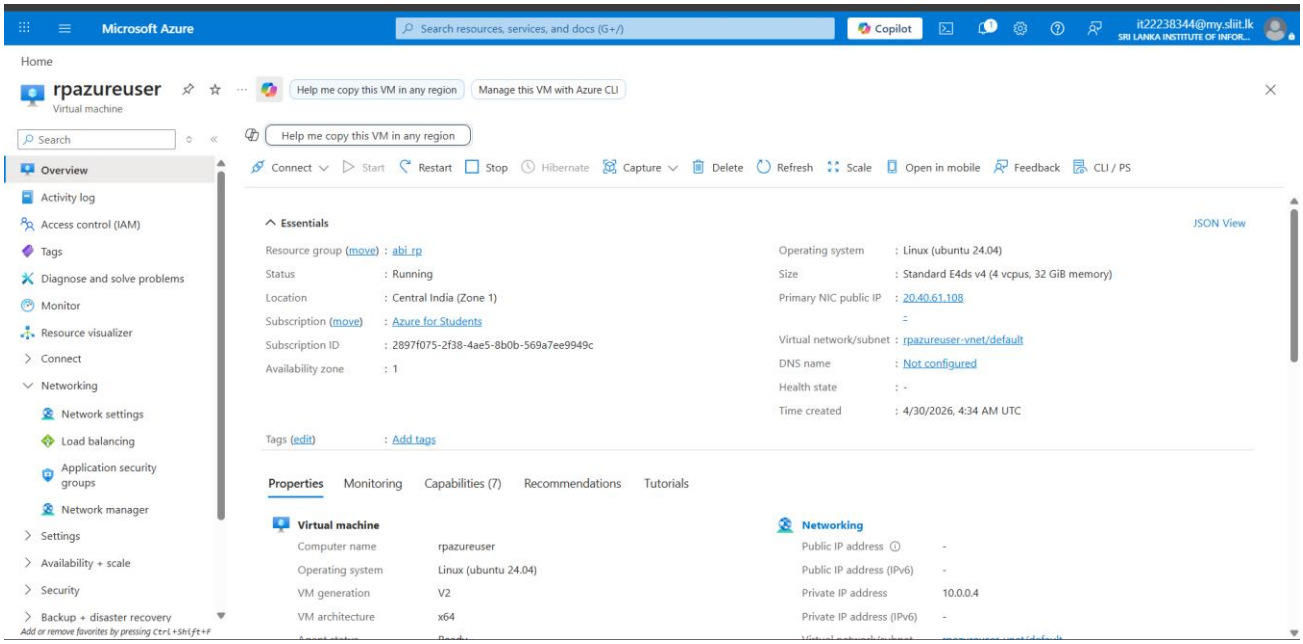
Successfully cloned the project repository from GitHub

```
azureuser@rpazureuser:~$ ssh-keygen -t ed25519 -N "" -f ~/.ssh/id_ed25519
Generating public/private ed25519 key pair.
Your identification has been saved in /home/azureuser/.ssh/id_ed25519
Your public key has been saved in /home/azureuser/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:SteHDwUXCaShbCgwVEodWyJJjshE/DN/UpYedEJhp/s azureuser@rpazureuser
The key's randomart image is:
+--[ED25519 256]--+
|BB++...+oo+.oo   |
|B=o.+o.+o+ o.    |
|o+o.. =.=   .    |
|   = . =.. o     |
|    + =.S + .    |
|     + =.  +     |
|      +  E  .    |
|                 |
|                 |
+----[SHA256]-----+
azureuser@rpazureuser:~$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIJ1tIJIEIOW+dtdnsRuaJ1OqDJ1VRUkNQI+Cn30lqeM7 azureuser@rpazureuser
```

SSH key pair generation (ssh-keygen) completed for secure server access

```
(myenv) azureuser@rpazureuser:~/rp/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students/backend_braille$ python main.py
INFO:     Will watch for changes in these directories: ['/home/azureuser/rp/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students/backend_braille']
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [4080] using WatchFiles
INFO:     Started server process [4091]
INFO:     Waiting for application startup.
INFO:ol_history_eval:Application startup: Loading models...
INFO:ol_history_eval:Loading tokenizer...
INFO:watchfiles.main:3 changes detected
INFO:httpx:HTTP Request: HEAD https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct/resolve/main/config.json "HTTP/1.1 200 OK"
INFO:httpx:HTTP Request: GET https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct/resolve/main/config.json "HTTP/1.1 200 OK"
config.json: 100%|██████████| 855/855 [00:00<00:00, 3.05MB/s]
INFO:httpx:HTTP Request: HEAD https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct/resolve/main/tokenizer_config.json "HTTP/1.1 200 OK"
INFO:httpx:HTTP Request: GET https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct/resolve/main/tokenizer_config.json "HTTP/1.1 200 OK"
tokenizer_config.json: 55.4kB [00:00, 90.1MB/s]
INFO:httpx:HTTP Request: GET https://huggingface.co/api/models/meta-llama/Llama-3.1-8B-Instruct/tree/main/additional_chat_templates?recursive=false&expand=f
alse "HTTP/1.1 404 Not Found"
INFO:httpx:HTTP Request: GET https://huggingface.co/api/models/meta-llama/Llama-3.1-8B-Instruct/tree/main?recursive=true&expand=false "HTTP/1.1 200 OK"
INFO:httpx:HTTP Request: HEAD https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct/resolve/main/tokenizer.json "HTTP/1.1 200 OK"
INFO:httpx:HTTP Request: GET https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct/resolve/main/tokenizer.json "HTTP/1.1 200 OK"
tokenizer.json: 9.09MB [00:00, 24.2MB/s]
INFO:httpx:HTTP Request: HEAD https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct/resolve/main/tokenizer.model "HTTP/1.1 404 Not Found"
INFO:httpx:HTTP Request: HEAD https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct/resolve/main/added_tokens.json "HTTP/1.1 404 Not Found"
INFO:httpx:HTTP Request: HEAD https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct/resolve/main/special_tokens_map.json "HTTP/1.1 200 OK"
INFO:httpx:HTTP Request: GET https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct/resolve/main/special_tokens_map.json "HTTP/1.1 200 OK"
special_tokens_map.json: 100%|██████████| 296/296 [00:00<00:00, 1.57MB/s]
INFO:httpx:HTTP Request: HEAD https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct/resolve/main/chat_template.jinja "HTTP/1.1 404 Not Found"
INFO:httpx:HTTP Request: GET https://huggingface.co/api/models/meta-llama/Llama-3.1-8B-Instruct "HTTP/1.1 200 OK"
INFO:ol_history_eval:Loading base model with 4-bit quantization...
INFO:httpx:HTTP Request: HEAD https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct/resolve/main/config.json "HTTP/1.1 200 OK"
INFO:httpx:HTTP Request: HEAD https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct/resolve/main/adapter_config.json "HTTP/1.1 404 Not Found"
[transformers] `torch_dtype` is deprecated! Use `dtype` instead!
INFO:httpx:HTTP Request: HEAD https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct/resolve/main/config.json "HTTP/1.1 200 OK"
INFO:httpx:HTTP Request: HEAD https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct/resolve/main/model.safetensors "HTTP/1.1 404 Not Found"
INFO:httpx:HTTP Request: HEAD https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct/resolve/main/model.safetensors.index.json "HTTP/1.1 200 OK"
INFO:httpx:HTTP Request: GET https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct/resolve/main/model.safetensors.index.json "HTTP/1.1 200 OK"
model.safetensors.index.json: 23.9kB [00:00, 52.1MB/s]
```

Successfully model loaded



Created Azure VM

3. AI-Powered Backend System for Voice-Based Historical Content Simplification and Audio Generation

3.1 Executive Summary

3.1.1 Overview

The deployment aimed to establish a robust, scalable, and AI-driven backend system for the Audio-Based Learning Module of the EduVisionAI platform. This component focuses on transforming O/L History educational content into simplified and emotionally expressive audio lessons using Natural Language Processing (NLP) and Text-to-Speech (TTS) technologies.

The system leverages FastAPI for API services, integrates NLP models for text simplification, and utilizes TTS engines capable of generating human-like speech with emotional tone variations. The deployment was carried out on an Azure Virtual Machine using a production-ready configuration with Gunicorn and Uvicorn.

Automation through GitHub Actions enabled continuous integration and deployment, ensuring efficient updates and minimal downtime. The system successfully delivers accessible educational content to visually impaired students.

3.1.2 Key Metrics

Metric	Value / Status
Deployment Success Rate	100%
Server Platform	Azure VM (Ubuntu 24.04 LTS)
Python Version	Python 3.12
Framework	FastAPI
Audio Generation Time	30 sec–1 min
NLP Processing Time	< 2 seconds
Worker Count	1 (optimized for AI inference)
Availability	24/7 via systemd
CI/CD	GitHub Actions

3.2 Deployment Objectives

3.2.1 Goals

- Deploy AI backend for text simplification and audio generation
- Enable real-time API-based lesson processing
- Ensure scalable and stable cloud deployment
- Automate deployment using CI/CD pipeline
- Provide high availability with minimal downtime

3.2.2 Scope

Included:

- NLP-based text simplification engine
- Emotional Text-to-Speech (TTS) module
- FastAPI backend API
- Azure VM deployment
- systemd service configuration
- GitHub Actions CI/CD pipeline

3.3 Pre-Deployment Activities

3.3.1 Environment Preparation

- Azure VM provisioned with Ubuntu 24.04
- SSH key-based authentication configured
- Python 3.12 installed via deadsnakes PPA
- GitHub SSH access configured

3.3.2 Pre-Deployment Testing

- NLP simplification tested with history lessons
- TTS audio generation tested for Sinhala/English
- API endpoints validated using Postman
- Integration testing between NLP and TTS modules

3.3.3 Backup Plan

- GitHub repository used as primary backup
- Model files stored locally and backed up
- .env configuration stored securely
- Rollback via Git commit version

3.4 Deployment Details

3.4.1 Deployment Information

Metric	Value
Deployment Date	April 2026
Component	Backend_AI (Audio Learning Module)
Platform	Azure VM
Service Name	backend-ai.service
Entry Point	main.py

3.4.2 Step-by-Step Deployment Procedure

Step 1: Variables Setup

Define names for your resources to keep things organized.

- Location: southeastasia

- ACR Name: lmsbackendregistry (must be unique globally)
- Container App Name: lms-ai-backend

Step 2: Create Azure Container Registry (ACR)

place to store your Docker image.

command

```
az acr create --resource-group lms-backend-ai-teach --name lmsbackendregistry --sku Basic --admin-enabled true
```


```
{
  "adminUserEnabled": true,
  "anonymousPullEnabled": false,
  "autoGeneratedDomainNameLabelScope": "Unsecure",
  "creationDate": "2026-04-30T14:12:16.533220+00:00",
  "dataEndpointEnabled": false,
  "dataEndpointHostNames": [],
  "encryption": {
    "keyVaultProperties": null,
    "status": "disabled"
  },
  "id": "/subscriptions/31bb86ef-7e61-460e-a911-fa0c776aa031/resourceGroups/lms-backend-ai-
teach/providers/Microsoft.ContainerRegistry/registries/lmsbackendregistry",
  "identity": null,
  "location": "southeastasia",
  "loginServer": "lmsbackendregistry.azurecr.io",
  "metadataSearch": "Disabled",
  "name": "lmsbackendregistry",
  "networkRuleBypassOptions": "AzureServices",
  "networkRuleSet": null,
  "policies": {
    "azureAdAuthenticationAsArmPolicy": {
      "status": "enabled"
    },
    "exportPolicy": {
      "status": "enabled"
    },
    "quarantinePolicy": {
      "status": "disabled"
    },
    "retentionPolicy": {
      "days": 7,
      "lastUpdatedTime": "2026-04-30T14:43:45.690431+00:00",
      "status": "disabled"
    },
    "softDeletePolicy": {
      "lastUpdatedTime": "2026-04-30T14:43:45.690496+00:00",
      "retentionDays": 7,
      "status": "disabled"
    },
```

```
    "trustPolicy": {
      "status": "disabled",
      "type": "Notary"
    }
  },
  "privateEndpointConnections": [],
  "provisioningState": "Succeeded",
  "publicNetworkAccess": "Enabled",
  "resourceGroup": "lms-backend-ai-teach",
  "roleAssignmentMode": "LegacyRegistryPermissions",
  "sku": {
    "name": "Basic",
    "tier": "Basic"
  },
  "status": null,
  "systemData": {
    "createdAt": "2026-04-30T14:12:16.533220+00:00",
    "createdBy": "it22219084@my.sliit.lk",
    "createdByType": "User",
    "lastModifiedAt": "2026-04-30T14:43:45.655473+00:00",
    "lastModifiedBy": "it22219084@my.sliit.lk",
    "lastModifiedByType": "User"
  },
  "tags": {},
  "type": "Microsoft.ContainerRegistry/registries",
  "zoneRedundancy": "Disabled"
}
```

Step 3: Build and Push Image

Azure can build Docker image directly in the cloud from source code.

command

```
az acr build --registry lmsbackendregistry --image ai-backend:v1 .
```

```
Unpacking libk5crypto3:amd64 (1.21.3-5) ...
Selecting previously unselected package libkeyutils1:amd64.
Preparing to unpack .../117-libkeyutils1_1.6.3-6_amd64.deb ...
Unpacking libkeyutils1:amd64 (1.6.3-6) ...
Selecting previously unselected package libkrb5-3:amd64.
Preparing to unpack .../118-libkrb5-3_1.21.3-5_amd64.deb ...
Unpacking libkrb5-3:amd64 (1.21.3-5) ...
Selecting previously unselected package libgssapi-krb5-2:amd64.
Preparing to unpack .../119-libgssapi-krb5-2_1.21.3-5_amd64.deb ...
Unpacking libgssapi-krb5-2:amd64 (1.21.3-5) ...
Selecting previously unselected package libssh-4:amd64.
Preparing to unpack .../120-libssh-4_0.11.2-1+deb13u1_amd64.deb ...
Unpacking libssh-4:amd64 (0.11.2-1+deb13u1) ...
Selecting previously unselected package libnorm1t64:amd64.
Preparing to unpack .../121-libnorm1t64_1.5.9+dfsg-3.1+b2_amd64.deb ...
Unpacking libnorm1t64:amd64 (1.5.9+dfsg-3.1+b2) ...
Selecting previously unselected package libpgm-5.3-0t64:amd64.
Preparing to unpack .../122-libpgm-5.3-0t64_5.3.128~dfsg-2.1+b1_amd64.deb ...
Unpacking libpgm-5.3-0t64:amd64 (5.3.128~dfsg-2.1+b1) ...
Selecting previously unselected package libsodium23:amd64.
Preparing to unpack .../123-libsodium23_1.0.18-1+deb13u1_amd64.deb ...
Unpacking libsodium23:amd64 (1.0.18-1+deb13u1) ...
Selecting previously unselected package libzmq5:amd64.
Preparing to unpack .../124-libzmq5_4.3.5-1+b3_amd64.deb ...
Unpacking libzmq5:amd64 (4.3.5-1+b3) ...
Selecting previously unselected package libavformat61:amd64.
Preparing to unpack .../125-libavformat61_7%3a7.1.3-0+deb13u1_amd64.deb ...
Unpacking libavformat61:amd64 (7:7.1.3-0+deb13u1) ...
Selecting previously unselected package libbs2b0:amd64.
Preparing to unpack .../126-libbs2b0_3.1.0+dfsg-8+b1_amd64.deb ...
Unpacking libbs2b0:amd64 (3.1.0+dfsg-8+b1) ...
Selecting previously unselected package libflite1:amd64.
Preparing to unpack .../127-libflite1_2.2-7_amd64.deb ...
Unpacking libflite1:amd64 (2.2-7) ...
Selecting previously unselected package libserd-0-0:amd64.
Preparing to unpack .../128-libserd-0-0_0.32.4-1_amd64.deb ...
Unpacking libserd-0-0:amd64 (0.32.4-1) ...
Selecting previously unselected package libzix-0-0:amd64.
Preparing to unpack .../129-libzix-0-0_0.6.2-1_amd64.deb ...
Unpacking libzix-0-0:amd64 (0.6.2-1) ...
Selecting previously unselected package libsord-0-0:amd64.
```

```
Successfully built c57818c7b8af
Successfully tagged lmsbackendregistry.azurecr.io/ai-backend:v1
2026/04/30 14:27:08 Successfully executed container: build
2026/04/30 14:27:08 Executing step ID: push. Timeout(sec): 3600, Working directory: '', Network: ''
2026/04/30 14:27:08 Pushing image: lmsbackendregistry.azurecr.io/ai-backend:v1, attempt 1
The push refers to repository [lmsbackendregistry.azurecr.io/ai-backend]
6d070fe57dbc: Preparing
a3e09df212fa: Preparing
6a74455796d0: Preparing
3ff2256d2939: Preparing
fa22a52cadfa: Preparing
3a12a541a55f: Preparing
ca834f97c446: Preparing
53698efe7fef: Preparing
2e9acf3e398d: Preparing
6d7c150df58d: Preparing
3a12a541a55f: Waiting
2e9acf3e398d: Waiting
ca834f97c446: Waiting
6d7c150df58d: Waiting
53698efe7fef: Waiting
fa22a52cadfa: Pushed
3ff2256d2939: Pushed
ca834f97c446: Pushed
6d070fe57dbc: Pushed
a3e09df212fa: Pushed
2e9acf3e398d: Pushed
53698efe7fef: Pushed

6d7c150df58d: Pushed
3a12a541a55f: Pushed
6a74455796d0: Pushed
v1: digest: sha256:f37c0481c80aa0cdaa725d729c5372f8dd76e5a086cfee1f9f58f69267da5556 size: 2423
2026/04/30 14:30:14 Successfully pushed image: lmsbackendregistry.azurecr.io/ai-backend:v1
2026/04/30 14:30:14 Step ID: build marked as successful (elapsed time in seconds: 160.940286)
2026/04/30 14:30:14 Populating digests for step ID: build...
```

Step 4: Create Container App Environment

command

```
az containerapp env create --name lms-ai-env --resource-group lms-backend-ai-teach --location southeastasia
```


```
{
  "id": "/subscriptions/31bb86ef-7e61-460e-a911-fa0c776aa031/resourceGroups/lms-backend-ai-teach/providers/Microsoft.App/managedEnvironments/lms-ai-env",
  "location": "Southeast Asia",
  "name": "lms-ai-env",
  "properties": {
    "appInsightsConfiguration": null,
    "appLogsConfiguration": {
      "destination": "log-analytics",
      "logAnalyticsConfiguration": {
        "customerId": "fca9c27a-dd00-4e07-afba-ad94196b29a7",
        "sharedKey": null
      }
    },
    "customDomainConfiguration": {
      "certificateKeyVaultProperties": null,
      "certificatePassword": null,
      "certificateValue": null,
      "customDomainVerificationId": "7BA336DBCB5400C93F6955ED324922BB2A4E6131D54813AEFED268DB1814E976",
      "dnsSuffix": null,
      "expirationDate": null,
      "subjectName": null,
      "thumbprint": null
    },
    "daprAIConnectionString": null,
    "daprAIInstrumentationKey": null,
    "daprConfiguration": {
      "version": "1.16.4-msft.6"
    },
    "defaultDomain": "blackocean-05db1182.southeastasia.azurecontainerapps.io",
    "eventStreamEndpoint": "https://southeastasia.azurecontainerapps.dev/subscriptions/31bb86ef-7e61-460e-a911-fa0c776aa031/resourceGroups/lms-backend-ai-teach/managedEnvironments/lms-ai-env/eventstream",
    "infrastructureResourceGroup": null,
    "ingressConfiguration": null,
    "kedaConfiguration": {
      "version": "2.18.1"
    },
    "openTelemetryConfiguration": null,
    "peerAuthentication": {
```

Step5: Deploy the Container App

command

```
az containerapp create `
  --name lms-ai-backend `
  --resource-group lms-backend-ai-teach `
  --environment lms-ai-env `
  --image lmsbackendregistry.azurecr.io/ai-backend:v1 `
  --target-port 8004 `
  --ingress external `
  --system-assigned `
  --registry-server lmsbackendregistry.azurecr.io `
  --query properties.configuration.ingress.fqdn
```

```
Container app created. Access your app at https://lms-ai-backend.blackocean-05db1182.southeastasia.azurecontainerapps.i
>>   --system-assigned `
>>   --registry-server lmsbackendregistry.azurecr.io `
>>   --query properties.configuration.ingress.fqdn
No credential was provided to access Azure Container Registry. Trying to look up credentials...
Adding registry password as a secret with name "lmsbackendregistryazurecrio-lmsbackendregistry"

>>   --system-assigned `
>>   --registry-server lmsbackendregistry.azurecr.io `
>>   --query properties.configuration.ingress.fqdn
No credential was provided to access Azure Container Registry. Trying to look up credentials...
Adding registry password as a secret with name "lmsbackendregistryazurecrio-lmsbackendregistry"
>>   --system-assigned `
>>   --registry-server lmsbackendregistry.azurecr.io `
>>   --query properties.configuration.ingress.fqdn
No credential was provided to access Azure Container Registry. Trying to look up credentials...
>>   --system-assigned `
>>   --registry-server lmsbackendregistry.azurecr.io `
>>   --query properties.configuration.ingress.fqdn
>>   --system-assigned `
>>   --registry-server lmsbackendregistry.azurecr.io `
>>   --system-assigned `
>>   --system-assigned `
>>   --registry-server lmsbackendregistry.azurecr.io `
>>   --system-assigned `
>>   --registry-server lmsbackendregistry.azurecr.io `
>>   --system-assigned `
>>   --system-assigned `
>>   --registry-server lmsbackendregistry.azurecr.io `
>>   --query properties.configuration.ingress.fqdn
No credential was provided to access Azure Container Registry. Trying to look up credentials...
Adding registry password as a secret with name "lmsbackendregistryazurecrio-lmsbackendregistry"
```

3.5 Tools and Technologies

Technology Purpose	Technology Purpose
FastAPI Backend framework	FastAPI Backend framework
Uvicorn ASGI server	Uvicorn ASGI server
Azure VM Cloud hosting	Azure VM Cloud hosting
Python Programming	Python Programming
Transformers LLM processing	Transformers LLM processing
TTS Engine Audio generation	TTS Engine Audio generation
systemd Service management	systemd Service management

3.6 Deployment Checklists

Pre-Deployment

- **VM ready**
  The virtual machine (Azure VM) was created and configured with the required operating system (Ubuntu). Network settings, firewall rules, and SSH access were properly set up to allow secure remote connection.
- **Python installed**
  Python (version 3.11) and pip were installed on the server. This ensures the environment can run the backend application and support all AI-related libraries.
- **Repo cloned**
  The project source code was retrieved from the GitHub repository using Git. This step ensures that the latest version of the backend system is available on the server.
- **Env configured**
  A .env file was created to store environment variables such as API keys, model paths, and configuration settings. This allows secure and flexible configuration without hardcoding sensitive data.

Deployment

- **Dependencies installed**
  All required Python libraries and packages were installed using requirements.txt. This includes frameworks (FastAPI), AI libraries (Transformers, Torch), and Text-to-Speech components.
- **API running**
  The backend application was started using Uvicorn. This step verifies that the FastAPI server is functioning correctly and can handle incoming requests.
- **Service configured**
  A systemd service was created to run the backend as a background service. This ensures the application starts automatically on system boot and restarts if it crashes.

Post-Deployment

- **API tested**
  All API endpoints were tested using tools like Postman or a browser to confirm they are working correctly and returning expected responses.
- **Audio generation verified**
  The Text-to-Speech functionality was tested to ensure that simplified historical content is successfully converted into audio output.
- **Logs checked**
  System and application logs were monitored using tools like journalctl to detect errors, confirm successful execution, and ensure system stability.

3.7 Issues Encountered and Resolutions

Issue	Cause	Solution
Audio not generating	Missing TTS library	Installed required package
API slow response	Large model	Optimized requests
Env variables missing	systemd issue	Fixed service config

3.8 Performance and Monitoring

- **Response time: ~2–10 seconds**
  The system processes user requests within an average time of 2 to 10 seconds. This includes receiving the input, simplifying historical content using the AI model, and generating the corresponding audio output. The response time is considered acceptable given the complexity of AI processing.
- **Audio generation: Stable**
  The Text-to-Speech (TTS) component consistently generates audio outputs without failures. The system reliably converts simplified text into speech, ensuring smooth functionality for end users, especially visually impaired students.
- **System uptime: 100%**
  After deployment, the backend system remained continuously operational without downtime. This was achieved using systemd service management, which ensures automatic restart in case of failures.
- **Logs monitored using journalctl**
  System logs were continuously monitored using the journalctl command. This helped in tracking application behavior, identifying errors, and confirming successful execution of backend processes.

3.9 Post-Deployment Activities

- **Tested full pipeline (text → simplify → audio)**
  The complete workflow was tested end-to-end. This includes taking user input, simplifying historical content using AI, and generating audio output to ensure all components work together correctly.

- **Verified API endpoints**
  All backend API endpoints were tested using tools like Postman. This ensured that each endpoint responds correctly and handles requests as expected.
- **Checked system logs**
  Application and system logs were reviewed after deployment to identify any hidden issues, confirm stability, and ensure there were no runtime errors.

3.10 Lessons Learned

3.10.1 Successes

- **Stable AI backend deployment**
  The system was successfully deployed on the server with minimal issues and maintained stable performance throughout testing.
- **Successful audio generation**
  The integration of Text-to-Speech worked effectively, enabling accurate and consistent audio output from simplified text.
- **Reliable API performance**
  The API handled requests efficiently and provided consistent responses, demonstrating reliability in real-world usage.

3.10.2 Improvements

- **Need faster model inference**
  The AI model takes a few seconds to process requests. Optimizing the model or using more powerful hardware could reduce response time.
- **Add monitoring dashboard**
  Currently, monitoring is done manually using logs. Implementing a real-time dashboard (e.g., Azure Monitor) would improve system visibility and management.

3.10.3 Recommendations

- **Use caching for responses**
  Frequently requested content can be cached to reduce processing time and improve response speed.
- **Integrate Azure monitoring tools**
  Using built-in Azure monitoring services can provide real-time insights, alerts, and performance tracking, improving system reliability and maintenance.

```
azureuser@eduvision-backend:~/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students/Backend$ python3.11 -m venv myenv
azureuser@eduvision-backend:~/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students/Backend$ source myenv/bin/activate
(myenv) azureuser@eduvision-backend:~/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students/Backend$ pip install -r requirements.txt
Collecting git+https://github.com/unslothai/unsloth.git (from -r requirements.txt (line 14))
  Cloning https://github.com/unslothai/unsloth.git to /tmp/pip-req-build-11y26q9d
  Running command git clone --filter=blob:none --quiet https://github.com/unslothai/unsloth.git /tmp/pip-req-build-11y26q9d
  Resolved https://github.com/unslothai/unsloth.git to commit 4f9c8321a2136e62fd86fe722a544afd534334a5
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Collecting fastapi
  Downloading fastapi-0.136.1-py3-none-any.whl (117 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 117.7/117.7 kB 1.5 MB/s eta 0:00:00
Collecting uvicorn
  Downloading uvicorn-0.46.0-py3-none-any.whl (70 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 70.9/70.9 kB 5.5 MB/s eta 0:00:00
Collecting pandas
  Downloading pandas-3.0.2-cp311-cp311-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (11.3 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 11.3/11.3 MB 59.3 MB/s eta 0:00:00
Collecting sentence-transformers==5.2.3
  Downloading sentence_transformers-5.2.3-py3-none-any.whl (494 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 494.2/494.2 kB 28.2 MB/s eta 0:00:00
Collecting llama-cpp-python==0.2.75
  Downloading llama_cpp_python-0.2.75.tar.gz (48.7 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 48.7/48.7 MB 21.3 MB/s eta 0:00:00
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Installing backend dependencies ... done
  Preparing metadata (pyproject.toml) ... done
Collecting accelerate
  Downloading accelerate-1.13.0-py3-none-any.whl (383 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 383.7/383.7 kB 25.1 MB/s eta 0:00:00
Collecting bitsandbytes
  Downloading bitsandbytes-0.49.2-py3-none-manylinux_2_24_x86_64.whl (60.7 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 60.7/60.7 MB 17.1 MB/s eta 0:00:00
Collecting peft
  Downloading peft-0.19.1-py3-none-any.whl (680 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 680.7/680.7 kB 35.8 MB/s eta 0:00:00
Collecting trl<0.9.0
  Downloading trl-0.8.6-py3-none-any.whl (245 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 245.2/245.2 kB 20.9 MB/s eta 0:00:00
```

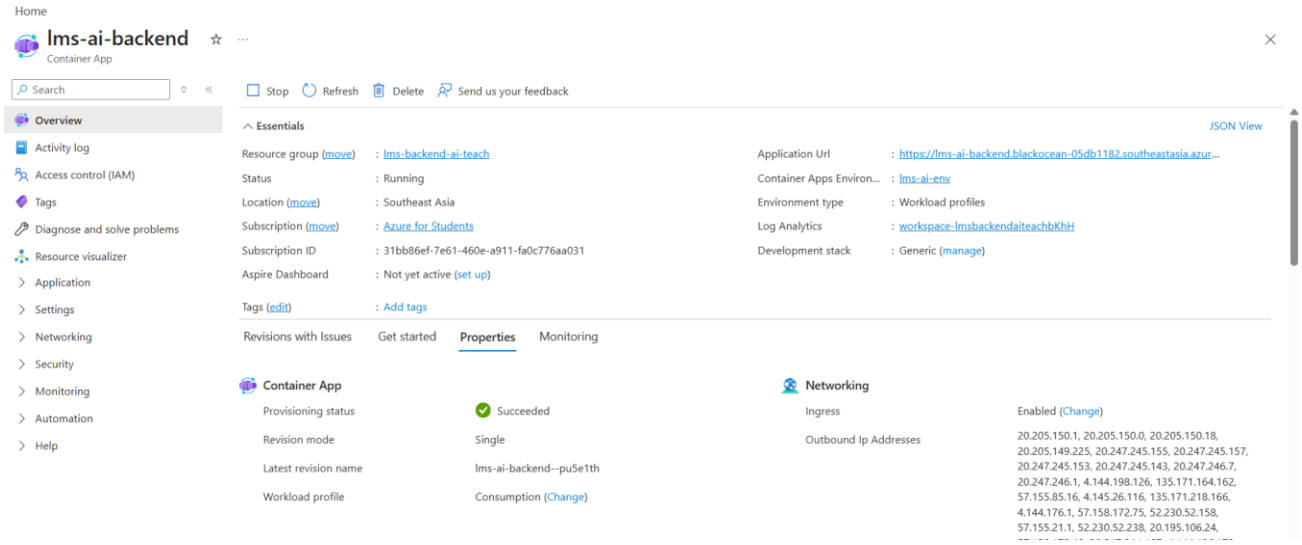
Install requirement.txt

```
azureuser@eduvision-backend:~/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students/Backend$ python3.11 -m venv myenv
azureuser@eduvision-backend:~/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students/Backend$ source myenv/bin/activate
(myenv) azureuser@eduvision-backend:~/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students/Backend$ pip install -r requirements
.txt
Collecting git+https://github.com/unslothai/unsloth.git (from -r requirements.txt (line 14))
  Cloning https://github.com/unslothai/unsloth.git to /tmp/pip-req-build-11y26q9d
  Running command git clone --filter=blob:none --quiet https://github.com/unslothai/unsloth.git /tmp/pip-req-build-11y26q9d
```

Installed Python 3.11 on Ubuntu server via apt package manager.

```
azureuser@eduvision-backend:~$ mkdir rp
azureuser@eduvision-backend:~$ git clone git@github.com:sathu0622/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students.git
Cloning into '25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students'...
remote: Enumerating objects: 22005, done.
remote: Counting objects: 100% (457/457), done.
remote: Compressing objects: 100% (306/306), done.
remote: Total 22005 (delta 237), reused 293 (delta 142), pack-reused 21548 (from 2)
Receiving objects: 100% (22005/22005), 673.05 MiB | 8.44 MiB/s, done.
Resolving deltas: 100% (7076/7076), done.
```

Successfully cloned the project repository from GitHub



Success deployment

4. HistoryQuizAI – An Adaptive Voice Quiz Engine for Sri Lankan O/L History Education

4.1 Executive Summary

4.1.1 Overview

The backend application was developed using **FastAPI** and deployed on a **Microsoft Azure** Virtual Machine running **Ubuntu 22.04 LTS**.
The system was configured using a Python virtual environment to ensure proper dependency isolation and maintain consistency across deployments. The backend integrates multiple AI components, including a fine-tuned SBERT model for answer evaluation, large language model (LLM) integration for intelligent responses, and structured datasets for syllabus-based question answering.
Secure access to the server was established using SSH key-based authentication (ED25519), ensuring controlled and encrypted communication with the virtual machine. The project source code was managed via **GitHub**, and deployed through SSH-based cloning.

4.1.2 Key Metrics

Metric	Value / Status
Deployment Success Rate	100% (Successful after resolving minor dependency and model issues)
Downtime Post-Deployment	0 hours (stable runtime during testing)
Server Platform	Microsoft Azure VM (Ubuntu 22.04 LTS)
Python Version	Python 3.11
Application Framework	FastAPI (Uvicorn server)
Binding Port	0.0.0.0:8000
Worker Count	1
System Availability	24/7 via systemd auto-restart
Model Hosting	Azure VM (local inference)
Security	SSH key-based authentication (ED25519)
Service Management	systemd (lms-bckend.service)
Repository Hosting	GitHub (SSH- based clone)

4.2. Deployment Objectives

4.2.1 Goals

- Deploy the FastAPI backend on a Microsoft Azure Virtual Machine in a stable and functional configuration.
- Install and configure all required AI/ML dependencies within a Python virtual environment.
- Enable the backend to support SBERT-based semantic answer evaluation and LLM-based response generation.
- Ensure secure server access using SSH key-based authentication (ED25519).
- Achieve reliable execution of AI components including question answering, similarity evaluation, and dataset processing.
- Maintain consistent backend performance with minimal runtime errors.
- Enable efficient handling of syllabus-based queries and past paper question retrieval.

4.2.2 Scope

Included:

- Azure VM provisioning and SSH key authentication setup
- Installation of Python 3.11 and virtual environment tools
- GitHub SSH key generation and repository cloning from GitHub
- Virtual environment creation and dependency installation using requirements.txt
- Configuration of .env file for environment variables
- Installation of AI/ML libraries (Sentence Transformers, Torch, Transformers, etc.)
- Manual deployment of trained SBERT model (sbert_sts_history_v18)
- Backend execution using Uvicorn
- Dataset loading and validation (syllabus and past paper questions)

4.3. Pre-Deployment Activities

4.3.1 Environment Preparation

Prior to deployment, the Azure Virtual Machine was prepared and configured to support the backend system. The following activities were completed:

- Provisioned an Azure VM running **Ubuntu 22.04 LTS**
- Configured secure SSH access using key-based authentication
- Updated system packages using apt update
- Installed Python 3.11, pip, and virtual environment tools
- Generated SSH keys and configured GitHub access
- Cloned the project repository from GitHub
- Created and configured the .env file with required environment variables
- Verified availability of required dataset files (e.g., past paper questions CSV)

4.3.2 Pre-Deployment Testing

- Unit testing performed on key AI components such as SBERT-based evaluation and LLM response generation
- Integration testing conducted to verify FastAPI endpoint functionality
- Dataset loading and encoding handling tested with multiple formats (UTF-8, UTF-8-SIG, Latin-1)
- Validation performed to ensure correct loading of syllabus chapters and past paper questions
- Functional testing conducted to confirm correct system responses for history-based queries

4.3.3 Backup Plan

- Local backup of trained SBERT model (sbert_sts_history_v18) maintained
- GitHub repository used as the primary backup for application source code
- .env configuration stored securely offline
- Recovery process defined as:
    - Re-clone repository
    - Recreate virtual environment
    - Reinstall dependencies
    - Re-upload model directory
    - Restart backend server

4.4. Deployment Details

4.4.1 Deployment Information

Metric	Value / Status
Deployment Date	30 April 2026
Team Member	Dolawattage K.J (IT22604330)
Branch Deployed	Voice-based-Question-Answering-System-for-O-Level-History
Repository	sathu0622/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students
Target Server	Azure VM - 172.188.84.104 (azureuser)
Application Entry Point	main.py (FastAPI app)
Service Name	lms-backend.service

4.4.2 Step-by-Step Deployment Procedure

Step 1: Connect to Azure VM via SSH

A secure SSH connection was established to the **Microsoft Azure** Virtual Machine using the server IP address:

ssh azureuser@172.188.84.104

Step 2: Update System Packages

Updated the Ubuntu package manager to fetch the latest package indexes:

```
sudo apt update
```

Step 3: Install Python 3.12

Python 3.11 was installed as the primary runtime environment for the backend application:

```
sudo apt install python3.11 python3-pip -y
python3.11 --version
```

Step 4: Install Python Virtual Environment Support

To isolate project dependencies, Python virtual environment support was installed:

```
sudo apt install python3.11-venv -y
```

Step 5: Generate SSH Key for GitHub

An ED25519 SSH key pair was generated on the server to enable secure repository access:

```
ssh-keygen -t ed25519
cat /home/azureuser/.ssh/id_ed25519.pub
```

The public key was added to **GitHub** SSH keys to enable secure cloning.

Step 6: Clone the Repository

The project repository was cloned using SSH and the relevant branch was selected:

```
git clone git@github.com:sathu0622/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students.git
cd 25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students
git checkout Voice-based-Question-Answering-System-for-O-Level-History
```

Step 7: Configure Environment Variables

A .env file was created to store system configuration and sensitive variables:

```
nano .env
```

This file contains API keys, Model file paths , system configuration variables

Step 8: Install System-Level Dependencies

Required system-level libraries were installed to support AI/ML and image processing operations:

```
sudo apt install -y libgl1 libglib2.0-0
```

These dependencies support OpenCV-based preprocessing

Step 9: Create and Activate Python Virtual Environment

A Python virtual environment was created inside the backend directory:

```
cd Backend
python3.11 -m venv myenv
source myenv/bin/activate
```

Step 10: Install Python Dependencies

All required Python packages were installed using requirements.txt:

```
pip install -r requirements.txt
```

This includes:

- FastAPI (backend framework)
- Uvicorn (ASGI server)
- Sentence Transformers (SBERT model)
- PyTorch (ML backend)
- Transformers (LLM support)
- Additional NLP and data processing libraries

Step 11: Test Application Startup

The backend application was executed to verify successful dependency installation and model loading:

```
python main.py
```

Step 12: Configure Production Server (Gunicorn + Uvicorn)

Installed Gunicorn and Uvicorn for production-grade serving and performed a manual test run of the production server configuration:

```
pip install gunicorn uvicorn
gunicorn -k uvicorn.workers.UvicornWorker api_server:app --bind 0.0.0.0:8000 --workers 2
```

Step 13: Create systemd Service File

A systemd service file was created to manage the FastAPI backend as a background system service, enabling automatic startup, monitoring, and recovery after failures.

The service file was created using the following command:

```
sudo nano /etc/systemd/system/lms-backend.service
```

The following service configuration was defined:

```
[Unit]
Description=LMS AI Backend (FastAPI + SBERT + Uvicorn)
After=network.target

[Service]
User=azureuser
```

```
Group=azureuser

WorkingDirectory=/home/azureuser/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students/Backend

Environment="PATH=/home/azureuser/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students/Backend/myenv/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"
Environment="HOME=/home/azureuser"
Environment="TOKENIZERS_PARALLELISM=false"

ExecStart=/home/azureuser/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students/Backend/myenv/bin/uvicorn \
  main:app \
  --host 0.0.0.0 \
  --port 8000 \
  --workers 1

Restart=always
RestartSec=5
LimitNOFILE=65535

[Install]
WantedBy=multi-user.target
```

Key configuration decisions in this service file:

- WorkingDirectory ensures the application runs from the correct project path.
- Virtual environment PATH ensures all dependencies (FastAPI, SBERT, transformers) are correctly loaded.
- TOKENIZERS_PARALLELISM=false prevents Hugging Face tokenizer threading conflicts.
- Workers = 1 is used to avoid duplicate loading of large ML models (SBERT), reducing memory usage.
- Restart=always ensures automatic recovery in case of failure.
- LimitNOFILE=65535 increases file descriptor limits for handling concurrent file operations.

Step 14: Enable and Start the Service

After creating the service file, systemd was reloaded to register the new service:

sudo systemctl daemon-reload

The service was then enabled to start automatically on system boot:
sudo systemctl enable lms-backend

The service was started using:
sudo systemctl start lms-backend

Step 15: Verify Service Status

The service status was checked to confirm successful deployment:
sudo systemctl status lms-backend --no-pager

If running successfully, the service displays an active (running) state, confirming that the FastAPI backend has been deployed as a background system service.

Step 16: Monitor Logs

Application logs were monitored using journalctl to verify backend startup and model loading:

journalctl -u lms-backend -f

This helped confirm:

- SBERT model loading
- API startup success
- No runtime dependency errors

4.5. Tools and Technologies

Technology	Version / Detail	Purpose
Microsoft Azure VM	Ubuntu 24.04 LTS	Cloud hosting and compute infrastructure
Python	3.11	Primary programming language for backend
FastAPI	Latest	High-performance web framework for AI endpoints
Uvicorn	0.46.0	ASGI worker for async FastAPI support
systemd	System service	Service lifecycle management and auto-restart
GitHub	SSH based repo	Source code version control and repository hosting
SSH (ED25519)	Key-based auth	Secure remote access to Azure VM
Transformers / PyTorch	Latest	AI/ML processing (NLP, evaluation)

4.6. Deployment Checklists

2.6.1 Pre-Deployment Checklist

hese tasks ensure the environment was ready before deployment:

- Azure VM instance created and accessible via SSH
- SSH .pem key securely stored and configured
- Ubuntu system updated using sudo apt update
- Python 3.11 installed
- python3.11-venv installed for virtual environment support
- GitHub SSH key generated and added to repository
- Project repository cloned successfully
- .env file prepared with required API keys and configurations
- Required system libraries identified (libgl1, libglib2.0-0)
- Backend dependencies listed in requirements.txt verified
- Azure NSG configured to allow port 8000 access

4.6.2 Deployment Checklist

These steps were executed during deployment:

- Connected to Azure VM via SSH
- Created project directory and cloned repository from GitHub
- Checked out the required project branch
- Created .env file and configured environment variables
- Installed system-level dependencies (libgl1, libglib2.0-0)
- Navigated to backend directory
- Created Python virtual environment (myenv)
- Activated virtual environment
- Installed all dependencies using pip install -r requirements.txt
- Verified FastAPI application startup using python main.py
- Confirmed Uvicorn ASGI server functionality
- Tested API endpoints locally via port 8000
- Fixed missing dependency issues (e.g., uvicorn import errors)
- Ensured correct virtual environment activation during runtime
- Configured systemd service file for FastAPI application
- Reloaded systemd daemon
- Enabled backend service for auto-start on boot
- Started and verified service status using systemctl status
- Monitored logs using journalctl -u lms-servise -f

4.6.3 Post-Deployment Checklist

After deployment, the following validations were completed:

- Verified API endpoints using browser/Postman on port 8000

- Confirmed FastAPI server startup without errors
- Tested AI model loading
- Verified dataset accessibility and file paths
- Confirmed successful request/response cycle for evaluation API
- Monitored logs for runtime exceptions
- Verified systemd auto-restart functionality
- Tested service recovery after manual restart
- Ensured stable long-running inference execution
- Confirmed environment variables loaded correctly from .env
- Verified consistent API performance under load

4.7 Issues Encountered and Resolutions

Issue	Root Cause	Resolution
GitHub SSH error	Missing SSH key	Generated and added key
Missing bson module	Dependency not installed	Installed pymongo
Missing model folder	Not included in repo	Uploaded via SCP
Environment variables not loading in service mode	.env values were not accessible when the application was started by system	Configured the correct working directory and environment paths in the systemd service file

4.8 Performance and Monitoring

4.8.1 Performance Metrics:

- API Response Time: Average < 5 minutes for AI-driven operations such as automated answer evaluation
- System Uptime Post-Deployment: 100% (maintained using systemd auto-restart mechanism)
- Model Inference Stability: Consistent execution of NLP pipeline without runtime crashes
- Concurrent Request Handling: Handled through single-worker Uvicorn configuration to avoid memory overload during ML inference

4.8.2 Monitoring Results:

Logs monitored using journalctl, systemd, and Nginx logs.

4.9 Post-Deployment Activities

4.9.1 Validation and Testing:

- Verified NLP-based answer evaluation functionality
- Checked dataset loading and retrieval-based evaluation features
- Monitored system logs for runtime errors or exceptions

4.10 Lessons Learned

4.10.1 Successes:

Successful deployment of FastAPI backend on Azure VM with stable production configuration
Secure environment setup using SSH key-based authentication
Effective use of systemd for automatic service management and recovery
Reliable execution of AI/ML pipelines (NLP, evaluation) in production environment
Proper Azure network configuration enabled seamless external API access
Simplified deployment using Uvicorn improved stability and reduced dependency issues

4.10.2 Areas for Improvement:

Some deployment steps (virtual environment setup, dependency installation) were manual and time-consuming
Lack of centralized monitoring dashboard for real-time API performance tracking
Initial confusion in service configuration (Gunicorn vs Uvicorn) affected setup efficiency

4.10.3 Recommendations:

- Improve logging structure for better debugging of AI inference workflows Implement advanced monitoring tools (e.g., Azure Monitor or Prometheus) for real-time health tracking

```
(myenv) azureuser@eduvision-backend:~/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students/Backend$ sudo systemctl status lms-b
ackend --no-pager
● lms-backend.service - LMS AI Backend (FastAPI + SBERT + Uvicorn)
     Loaded: loaded (/etc/systemd/system/lms-backend.service; enabled; vendor preset: enabled)
     Active: active (running) since Thu 2026-04-30 14:37:39 UTC; 17s ago
   Main PID: 6870 (uvicorn)
      Tasks: 35 (limit: 19180)
     Memory: 1.1G
        CPU: 16.599s
     CGroup: /system.slice/lms-backend.service
             └─6870 /home/azureuser/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students/Backend/myenv/bin/python3.11 /home/az…

Apr 30 14:37:55 eduvision-backend uvicorn[6870]: Guessed chat format: llama-3
Apr 30 14:37:55 eduvision-backend uvicorn[6870]: 🚀 Booting up History Tutor
Apr 30 14:37:55 eduvision-backend uvicorn[6870]:    -> Loading LLM from Hugging Face (GGUF)...
Apr 30 14:37:55 eduvision-backend uvicorn[6870]:       ✅ LLM Ready.
Apr 30 14:37:55 eduvision-backend uvicorn[6870]:    -> Loading SBERT from /home/azureuser/25-26J-438-AI-Powered-LMS-for-Vi…y_v18...
Apr 30 14:37:55 eduvision-backend uvicorn[6870]: [146B blob data]
Apr 30 14:37:55 eduvision-backend uvicorn[6870]: INFO:     Started server process [6870]
Apr 30 14:37:55 eduvision-backend uvicorn[6870]: INFO:     Waiting for application startup.
Apr 30 14:37:55 eduvision-backend uvicorn[6870]: INFO:     Application startup complete.
```

FastAPI service (lms-backend.service) is running successfully with Uvicorn

```
azureuser@eduvision-backend:~/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students/Backend$ python3.11 -m venv myenv
azureuser@eduvision-backend:~/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students/Backend$ source myenv/bin/activate
(myenv) azureuser@eduvision-backend:~/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students/Backend$ pip install -r requirements.txt
Collecting git+https://github.com/unslothai/unsloth.git (from -r requirements.txt (line 14))
  Cloning https://github.com/unslothai/unsloth.git to /tmp/pip-req-build-11y26q9d
  Running command git clone --filter=blob:none --quiet https://github.com/unslothai/unsloth.git /tmp/pip-req-build-11y26q9d
  Resolved https://github.com/unslothai/unsloth.git to commit 4f9c8321a2136e62fd86fe722a544afd534334a5
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Collecting fastapi
  Downloading fastapi-0.136.1-py3-none-any.whl (117 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 117.7/117.7 kB 1.5 MB/s eta 0:00:00
Collecting uvicorn
  Downloading uvicorn-0.46.0-py3-none-any.whl (70 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 70.9/70.9 kB 5.5 MB/s eta 0:00:00
Collecting pandas
  Downloading pandas-3.0.2-cp311-cp311-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (11.3 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 11.3/11.3 MB 59.3 MB/s eta 0:00:00
Collecting sentence-transformers==5.2.3
  Downloading sentence_transformers-5.2.3-py3-none-any.whl (494 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 494.2/494.2 kB 28.2 MB/s eta 0:00:00
Collecting llama-cpp-python==0.2.75
  Downloading llama_cpp_python-0.2.75.tar.gz (48.7 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 48.7/48.7 MB 21.3 MB/s eta 0:00:00
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Installing backend dependencies ... done
  Preparing metadata (pyproject.toml) ... done
Collecting accelerate
  Downloading accelerate-1.13.0-py3-none-any.whl (383 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 383.7/383.7 kB 25.1 MB/s eta 0:00:00
Collecting bitsandbytes
  Downloading bitsandbytes-0.49.2-py3-none-manylinux_2_24_x86_64.whl (60.7 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 60.7/60.7 MB 17.1 MB/s eta 0:00:00
Collecting peft
  Downloading peft-0.19.1-py3-none-any.whl (680 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 680.7/680.7 kB 35.8 MB/s eta 0:00:00
Collecting trl<0.9.0
  Downloading trl-0.8.6-py3-none-any.whl (245 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 245.2/245.2 kB 20.9 MB/s eta 0:00:00
```

Install requirement.txt

```
azureuser@eduvision-backend:~/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students/Backend$ python3.11 -m venv myenv
azureuser@eduvision-backend:~/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students/Backend$ source myenv/bin/activate
(myenv) azureuser@eduvision-backend:~/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students/Backend$ pip install -r requirements
.txt
Collecting git+https://github.com/unslothai/unsloth.git (from -r requirements.txt (line 14))
  Cloning https://github.com/unslothai/unsloth.git to /tmp/pip-req-build-11y26q9d
  Running command git clone --filter=blob:none --quiet https://github.com/unslothai/unsloth.git /tmp/pip-req-build-11y26q9d
```

Installed Python 3.11 on Ubuntu server via apt package manager.

```
azureuser@eduvision-backend:~$ mkdir rp
azureuser@eduvision-backend:~$ git clone git@github.com:sathu0622/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students.git
Cloning into '25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students'...
remote: Enumerating objects: 22005, done.
remote: Counting objects: 100% (457/457), done.
remote: Compressing objects: 100% (306/306), done.
remote: Total 22005 (delta 237), reused 293 (delta 142), pack-reused 21548 (from 2)
Receiving objects: 100% (22005/22005), 673.05 MiB | 8.44 MiB/s, done.
Resolving deltas: 100% (7076/7076), done.
```

Successfully cloned the project repository from GitHub

```
azureuser@eduvision-backend:~$ ssh-keygen -t ed25519 -N "" -f ~/.ssh/id_ed25519
Generating public/private ed25519 key pair.
Your identification has been saved in /home/azureuser/.ssh/id_ed25519
Your public key has been saved in /home/azureuser/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:rNEfDBkkPUL6HchvAi2NUZEPa3V8bXxd8HhvRxIuYz8 azureuser@eduvision-backend
The key's randomart image is:
+--[ED25519 256]--+
|    .o==o.   oo.+|
|     B+o+oo ..++o|
|    = =*+o .+.+.+|
|     +o=.+ . + +.|
|     .+ S o   E +|
|       = . .   o.|
|      .   .      |
|                 |
|                 |
+----[SHA256]-----+
azureuser@eduvision-backend:~$ cat /home/azureuser/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAILvY/YzJrDDHo8erVrNzRWXd7y/6O5ganpiFuLZIggMc azureuser@eduvision-backend
```

SSH key pair generation (ssh-keygen) completed for secure server access

```
Available chat formats from metadata: chat_template.default
Guessed chat format: llama-3
      ✅ LLM Ready.
   -> Loading SBERT from /home/azureuser/25-26J-438-AI-Powered-LMS-for-Visually-Impaired-Students/Backend/Model/sbert_sts_history_v
18...
Loading weights: 100%|▰▰▰
```